LA SAGESSE UNIVERSITY

FACULTY OF ENGINEERING

Design and Implementation of an Autonomous UAV System for Rapid Disaster Reconnaissance

Thesis study to obtain a Bachelor of Engineering in

Mechatronic

Prepared by:

Wadih DAHROUGE-201900388

Amine BATROUNI-201900473

Houssam HAMDAN-201800229

Furn El Chebbak – Lebanon

2025

LA SAGESSE UNIVERSITY

FACULTY OF ENGINEERING

Jury members:	**Dr. Eng. Roy ABI ZEID DAOU**	Supervisor
---		President
---		Reviewer
---		Reviewer

Furn El Chebbak – Lebanon

2025

Acknowledgment

Abstract

In the last few years, the number of disasters has exponentially increased; floods, snowstorms, wildfires, massive destruction, … These phenomena pose a great risk on individuals, where minutes of earlier awareness could save lives. Traditional search and rescue operations have multiple limitations, including slow response due to traffic and risks to the rescue team. This work aims to enhance disaster management by designing and implementing an autonomous Unmanned Aerial Vehicle (UAV) system, designed for rapid response in disaster scenarios and for environmental monitoring in dangerous and hard-to-reach areas. Equipped with advanced technologies, environmental sensors, imaging technology, and predictive machine learning algorithms, the UAV can detect fire, locate victims, and perform real-time vital signs analysis. This design is modular and can be customized to adapt to various operational needs.

In a disaster scenario, the operator inputs geolocation coordinates into the ground station, which generates a flight path for the UAV. It then autonomously navigates to waypoints while avoiding obstacles in real-time during flight. The UAV streams real-time data back to the central command, enabling the rescuer to detect fire and victims with the help of two AI models. It provides critical situational awareness and enables decision-making. This transition from traditional manual surveillance and rescue missions to autonomous UAV reconnaissance operations accelerates early intervention, minimizes risks, and ensures better resource management.

However, building a successful system requires addressing several operational challenges. The project faces multiple technical constraints: limited battery capacity restricts flight endurance, while low-quality components can compromise performance and data accuracy. Additionally, the communication range between the UAV and the ground station poses reliability issues, and restricted GPS access may affect navigation. To ensure precise data collection, periodic calibration of sensors, imaging equipment, and navigation systems is essential. Furthermore, regular maintenance routines, including performance inspections and timely battery recharging, are critical to keeping the UAV operational for extended missions.

As for the non-technical constraint UAV system must comply with local and international aviation regulations, including those set by the Lebanese Civil Aviation Authority (LCAA), the FAA (Federal Aviation Administration), and EASA (European Union Aviation Safety Agency) for

safety considerations, ISO training, competency of drone pilots and drone maintenance. Moreover, the drone operator must have experience in drone to take control in cases of malfunctions.

To ensure the best practice, the system integrates several advanced components such as a flight controller equipped with GPS for autonomous navigation, a companion computer to collect data from environmental sensors, a LIDAR to ensure obstacle avoidance and another for safe landing, as well as a speed controller providing the required speed to the motor to lift, navigate, and stabilize the quadcopter. As for the software implementation, the drone uses Arducopter firmware for stabilization and motor control, Robot Operating System (ROS) middleware for data acquisition and transmission, and a web server serving as a ground station that receives all the data and has access to control the drone. The three elements communicate together using MAVROS and ROSbridge protocol.

Ensuring the stability of the system and the best performance, the UAV must be tested and validated. The testing included fire and victim detection through various environmental conditions. In addition, autonomous navigation and obstacle avoidance are validated using a customized mission in a controlled environment to prove functionality.

In conclusion, the integration of this modular UAV system into disaster response and climate change monitoring strategies offers huge potential. It enhances operational efficiency, public safety, and climate resilience, providing a cost-effective solution to Lebanon's growing challenges related to natural disasters and environmental monitoring.

KEYWORDS: Autonomous UAV, Search and rescue (SAR), Fire detection, victim localization, Robot Operating System (ROS), Ardupilote, Ground station, Geolocation-based navigation, Obstacle avoidance, Machine learning

Table of content

List of Figures

List of Tables

Chapter 1: General Introduction

This first chapter is designed to show a general introduction to the proposed project. The problem that led us to develop this project will be stated, as well as a research query. Additionally, a general overview of the project will be presented. This chapter will also present a block diagram showing the main parts of the developed system, the objectives of this project, the motivation, and limitations and constraints. It will end with a brief presentation of the report's content and structure.

1.1 Problem Statement

Unmanned Aerial Vehicles (UAVs), usually known as drones, have considerably evolved over the past few decades. Originally, these systems were made for military surveillance and target practice applications. The idea originated in the early 1900s. The first known use occurred during World War I, when the United States created the crude "flying bomb" known as the Kettering Bug in 1918 (Austin R. , 2010) (Gertler, 2012). However, with technological evolution, UAVs have become an essential tool in many fields and numerous industries such as agriculture, logistics, surveillance, environmental monitoring, and search and rescue operations. UAVs are known for their high ability to access and reach hard areas, perform autonomous tasks, and operate efficiently in a wide variety of environments. Figure 1 shows the first drone made.

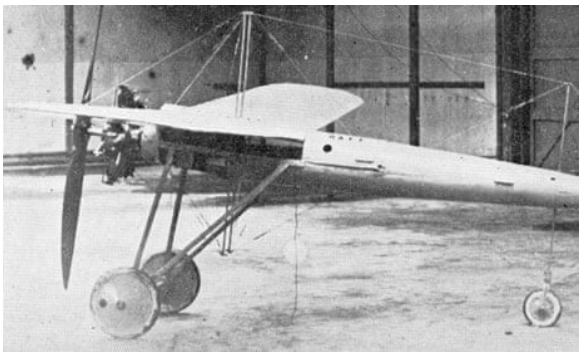

Figure 1 – First drone in the world (Sale, 2013)

In particular, autonomous UAVs, which can perform operations without human intervention, are now becoming popular and gaining support due to their ability to do complex tasks with precision and reliability. A study demonstrated that, with Artificial Intelligence's (AI)

rising, sensor technology improving, and navigation algorithm advancing, drones can be much more useful in deployment (Floreano, 2015). The evolution of UAVs from remote-controlled systems to autonomous drones made a big impact on the worldwide industrial and technological markets, and it's projected to grow exponentially in the coming years. One of the most critical applications of UAV technology is in Search and Rescue (SAR) missions and fire detection, where timely intervention is a matter of life and death. Natural disasters, wildfires, and emergency situations need a quick response and precise awareness to detect fire, locate victims, make damage assessment reports, and prevent further escalation (Waharte, 2010).

In the latest years, natural disasters have frequently increased, with over 400 recorded natural disasters each year (UCLouvain, 2024). Especially wildfires and snow-related emergencies have dramatically raised the challenges associated with search and rescue operations. Caused largely by climate change, these disasters have become more unpredictable and unstable, making timely and effective operations efforts more critical than ever (IPCC, 2021). Figure 2 represents the increase in recorded natural disasters over the last century.

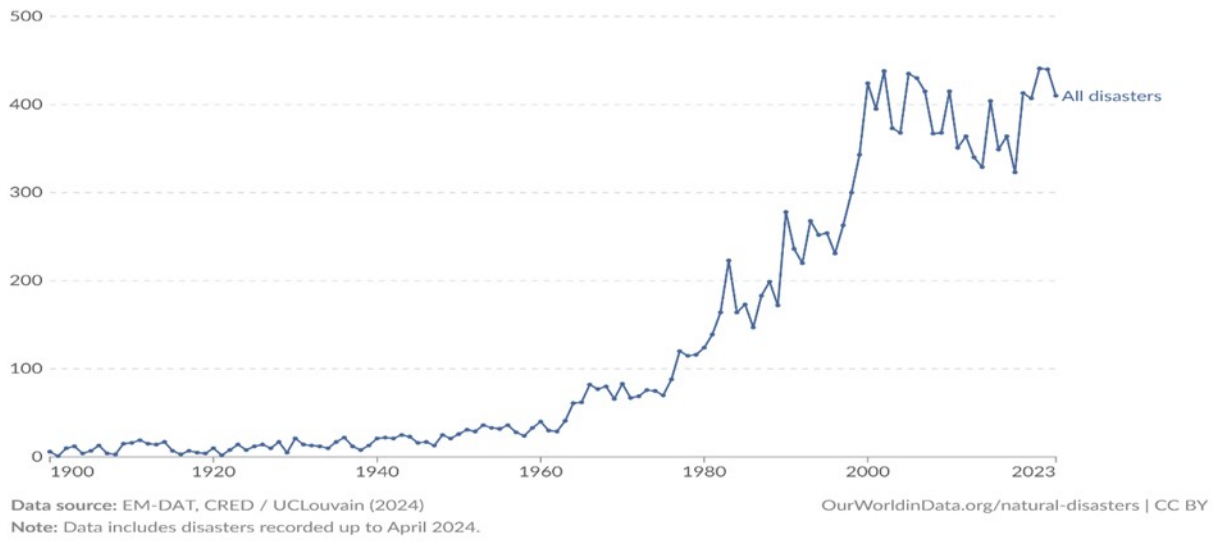

Figure 2 – Recorded natural disasters since 1900 (data, 2024)

Wildfires are becoming more destructive and common, constantly flooding large areas at fast speeds and causing dangerous environments packed with high temperatures, smoke, and toxic gases. These conditions intensely limit visibility and access, placing the lives of rescuers and casualties in danger (Johnston, Borchers-Arriagada, & Morgan, 2020). Research shows a massive increase in wildfires around the world in the last decade. For example, last year, wildfires burned

almost 367 million hectares worldwide, ranking as the 17th largest annual burned area since 2001. Moreover, according (Burgueño Salas, 2024), in the past three decades, the global death toll due to wildfires has exceeded 3,300. The global economic losses resulting from wildfire disasters in 2023 exceeded $10 billion (Statista, 2023). Finding stranded or bewildered victims in such dangerous situations becomes a pressing task. Victim localization is extremely difficult due to irregular wind patterns, terrain obstacles, and the complexity of fire behavior (Bowman & Williamson, 2017). Extreme weather events expose human and technological limits in conventional SAR techniques as they get more intense, leading to a change toward more sophisticated, automated solutions. Figure 3 illustrates the increase in forest fires in the past few decades in Portugal.

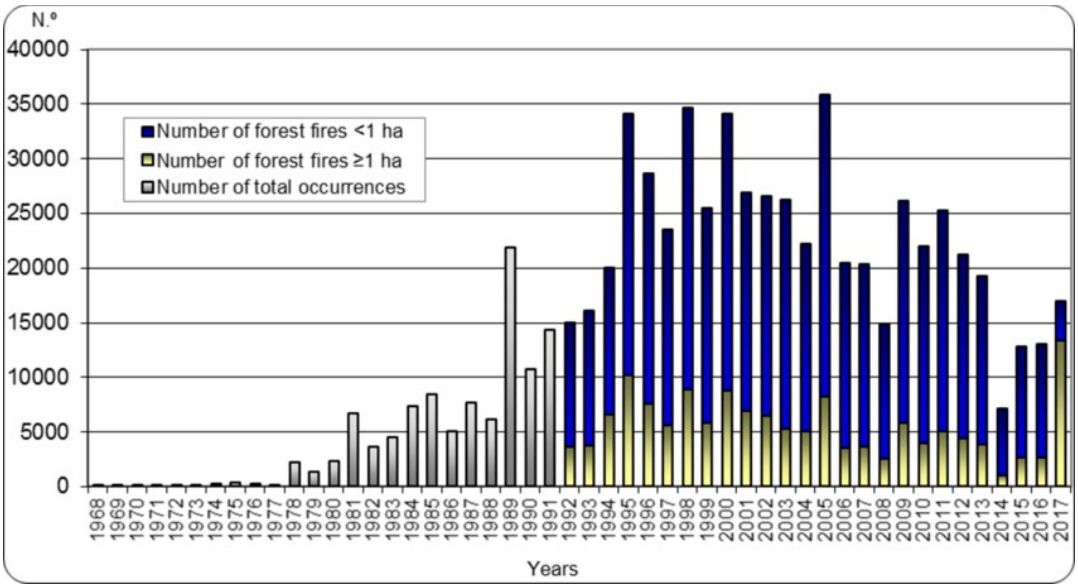

Figure 3 – Number of forest fires in Portugal each year (Lourenço, 2018)

Likewise, snow disasters, such as extreme cold snaps, blizzards, and avalanches, pose unusual search and rescue challenges. As victims may be buried under meters of snow, it is almost impossible to locate them using traditional methods like manual investigation or search dogs (Haegeli, Atkins, & Shandro, 2011). In 2024, avalanches killed at least 15 people in the US, including two skiers who perished in the Wasatch range of Utah following spring snowstorms. In addition, two enormous snowstorms influenced more than 290 million people in the United States (statistica, 2025). Besides, limited visibility, bad weather conditions, and vast mountainous terrains further complicate these operations (Jamieson & Schweizer, 2005). Since buried people are at a significant risk of hypothermia or suffocation within minutes of becoming trapped, time is of the essence during snow rescue operations.

One of the most critical concerns in snow emergencies and wildfire disasters is the victims' localization. Delayed or false detection can highly reduce survival rates. Traditional tools such as visual detection, GPS trackers, radio communication, usually fail in victim localization due to extreme conditions with obstruction, infrastructure damage, or interference (Hinkley, 2019). This critical limitation highlights the need for reliable, autonomous, and 24/7 systems capable of operating in hazardous and dangerous environments.

Recent research and field trials have shown that Unmanned Aerial Vehicles (UAVs) hold great promises for addressing these challenges. UAVs can quickly survey large, inaccessible areas while carrying thermal imaging, LiDAR, gas sensors, and computer vision systems to detect human activity and monitor fire or snow conditions (Tokekar & Isler, 2016) (Yuan, Wang, & Wang, 2020). Unlike human-led missions, UAVs reduce response time, increase safety, and provide adaptable platforms for multi-sensor integration and autonomous navigation (Adams & Friedland, 2011). These capabilities make UAV-based SAR systems an extremely effective and scalable solution for modern disaster response. Figure 4 shows the cost of winter storms in Europe each year.

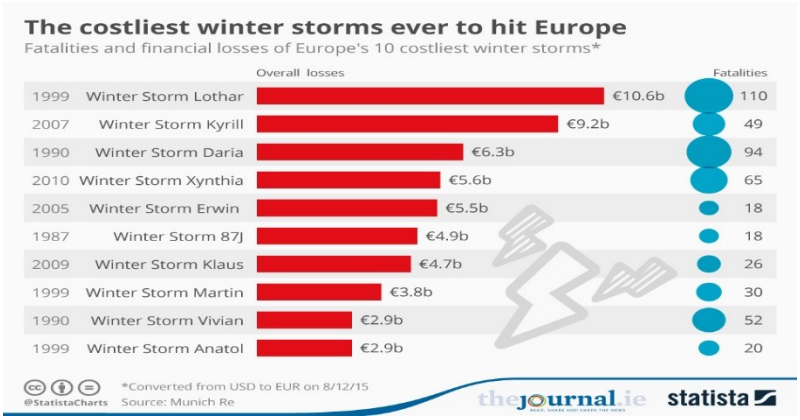

Figure 4 – Winter storm loses in Europe (statica, 2024)

The creation of an autonomous UAV system for SAR and fire detection becomes not only pertinent but also necessary in view of these expanding problems, massive number of victims, and enormous global economic losses. In situations involving wildfires and snow disasters, such a system can greatly improve the effectiveness, precision, and safety of rescue operations. Figure 5 presents some examples of disasters that happened during the past months.

(a) *(b)*

Figure 5 – Some disasters such as (a) California wildfire and (b) snow disaster (Press, 2024) (Hagler Geard, 2011)

1.2 Motivation

Nowadays, with the increase of natural disasters due to climate change, particularly in locations that humans cannot reach, autonomous UAVs are an essential need. The drone should operate in dynamic environments, with minimal human intervention, to contain the damage.

The motivation for designing and developing autonomous UAVs comes in many ways. Firstly, the desire to improve the efficiency of missions and protect the first responders. With UAVs, SAR will be much more accurate and achieve faster success in localization. A fast and aware response to the emergency will reduce the number of victims, or casualties, as well as nature and economic losses.

One of the most catastrophic scenarios that motivates our work is port Beirut explosions. On August 4, 2020, a massive explosion destroyed half of Beirut city with 2,750 tons of improperly stored ammonium nitrate. This explosion killed more than 250 people, injured over 7,000, and displaced 300,000 residents. The explosion caused massive damage to the infrastructure, with estimated losses of more than $15 billion. If a UAV system were launched into the region of fire at a rapid speed, we could prevent lots of damage via awareness provided to the disaster management team and operation in order to send the required reinforcement as needed to prevent the explosion. Secondly, using a low-cost autonomous system can reduce lots of resources and reinforcements that are not entirely needed, resulting in saving costs.

Lastly, this project contributes to the mechatronics field by addressing multiple challenges mainly autonomous decision-making, sensor data acquisition and transmission, and real-time

navigation. This project combines aerodynamics, mechatronics, and machine learning in one complete system. Figure 6 shows how the drone can detect fire and scan for a region affected by fire with a camera.

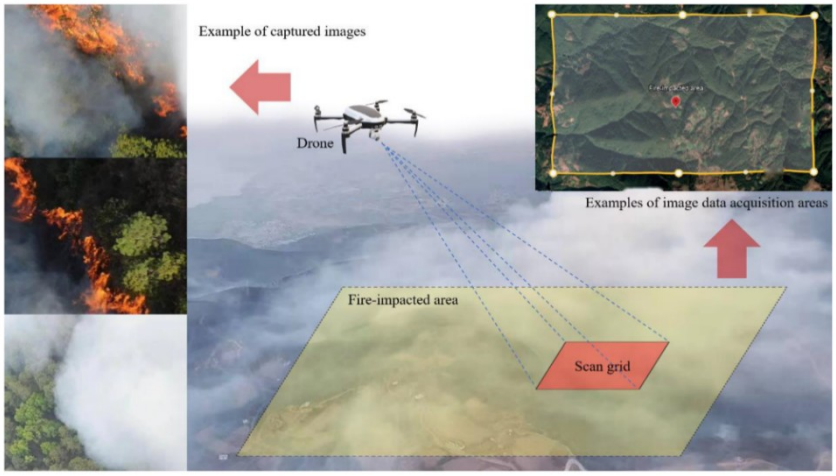

Figure 6 – Drone detections examples (Luan, 2024)

1.3 Objectives

Based on the concept presented in the introduction, the main objectives of this work are as follows:

- Design and develop an autonomous UAV capable of detecting fires and locating victims to support the Lebanese Civil Defense in firefighting and search and rescue operations;
- Operate effectively during disasters in hazardous or inaccessible areas where humans cannot easily reach, such as wildfires, snow-covered regions, and dense forests;
- Run in a fully autonomous mode, enabling the drone to navigate undefined environments and transmit alerts and data to the ground station without direct human intervention;
- Provide real-time, remote fire detection and situational awareness to enhance the effectiveness of emergency response teams;
- Avoid obstacles autonomously using LiDAR for truly independent operation;
- Implement a modular design that facilitates the integration of additional sensors, including thermal imaging systems when available;
- Generate a 3D map of the environment through camera-based photogrammetry to assist in damage assessment.

1.4 Specifications

After a meeting with the Lebanese Civil Defense, several specifications were placed to achieve the objective of this project:

- Autonomous navigation;
- Obstacle avoidance;
- Fire detection;
- Human detection;
- Spread fire prediction;
- Live video streaming;
- Environmental data;
- Long-range communication;
- Long flight duration;
- Bad weather condition suitable;
- Thermal imaging of the fire.

These objectives will help reduce the time to reach the fire and/or the patient while reducing risks and increasing the chances to save lifes.

1.5 Constraints

In this section, we will present the main constraints of this work. These constraints will be divided into technical and non-technical constraints.

1.5.1 Technical constraint

Many technical constraints and limitations exist based on engineering, design, and technological problems:

- *Short flight time:* Drones are limited by battery life because of the power consumed by the sensors, payload, and motors, which will lead to a short flight endurance estimated between 20 to 30 minutes;

- *Sensor accuracy and performance:* Most sensors in the market, including temperature and gas sensors, are low quality, which will affect the performance and the accuracy of reliably getting data;
- *Communication Range and problems:* Drones need to maintain a stable connection with the ground station all the time for control. The basic communication tools may be unstable, especially when it is needed to operate in remote areas;
- *GPS Dependency:* Many regions have limited GPS access, like dense forests, canyons, or snowy regions, and this will cause traditional waypoint navigation to fail;
- *Autonomous Navigation:* obstacle avoidance, and mission planning require reliable and accurate algorithms;
- *Processing Power:* high processing power for detection and image processing;
- *Weather Conditions:* Drones are sensitive to the wind and rain; therefore, it is a challenge to operate in all weathers;
- *Night Operation:* Night operation needs better camera, like infrared vision, to keep all advertised functionalities.

1.5.2 Non-technical constraints

In addition to technical constraints, there are servals non-technical limitations and constraints that reside in this work:

- *Legal and Regulatory Barriers*: Drone operators need legal authorization to fly, which require paperwork and documents. It must comply with the Federal Aviation Administration, the European Union Aviation Safety Agency, ISO, and IEC;
- *Budget Constraints*: UAVs with such requirements and features require a big budget to be reliably deployed long term and the use of better-quality sensors;
- *Short time*: This complex system needs time for development, testing, facing a lot of limitations within academic deadlines;
- *Ethical Considerations*: Mainly considered when flying over populated areas or using cameras;
- ***Team Skills*: This system integrates principles from mechatronics, aerodynamics, embedded systems, and software development. Although full artificial intelligence (AI) integration was not implemented in this version due to limited time and**

computational resources, the architecture was prepared with AI capabilities in mind. Early experiments with object detection models were also explored for future extension. As a result, the project requires a multidisciplinary team with knowledge in mechanical design, electronics, programming, and data processing.;

- *Location and Environment*: Real-world conditions like mountains, forests, snow, or smoke may not be easy to simulate or test during development.
- *Maintenance and Repair*: In field operations, such as in disaster zones, replacing parts, support tools, or repairing damage to the drone may be unavailable.

1.6 Block diagram

The proposed solution is an autonomous UAV system developed to support search and rescue operations, with a primary focus on fire detection and victim localization. Its high-level architecture is organized around three core subsystems: the aerial platform, the ground control station, and the communication link between them.

The UAV subsystem executes autonomous missions while collecting environmental data, capturing real-time video, and processing information locally. The ground control station offers mission planning tools and live monitoring through an interactive interface, enabling the operator to supervise operations remotely. The communication system ensures reliable data exchange between the UAV and the ground station, supporting real-time decision-making during missions.

This modular and scalable architecture enables the integration of future features and ensures adaptability across various operational scenarios.

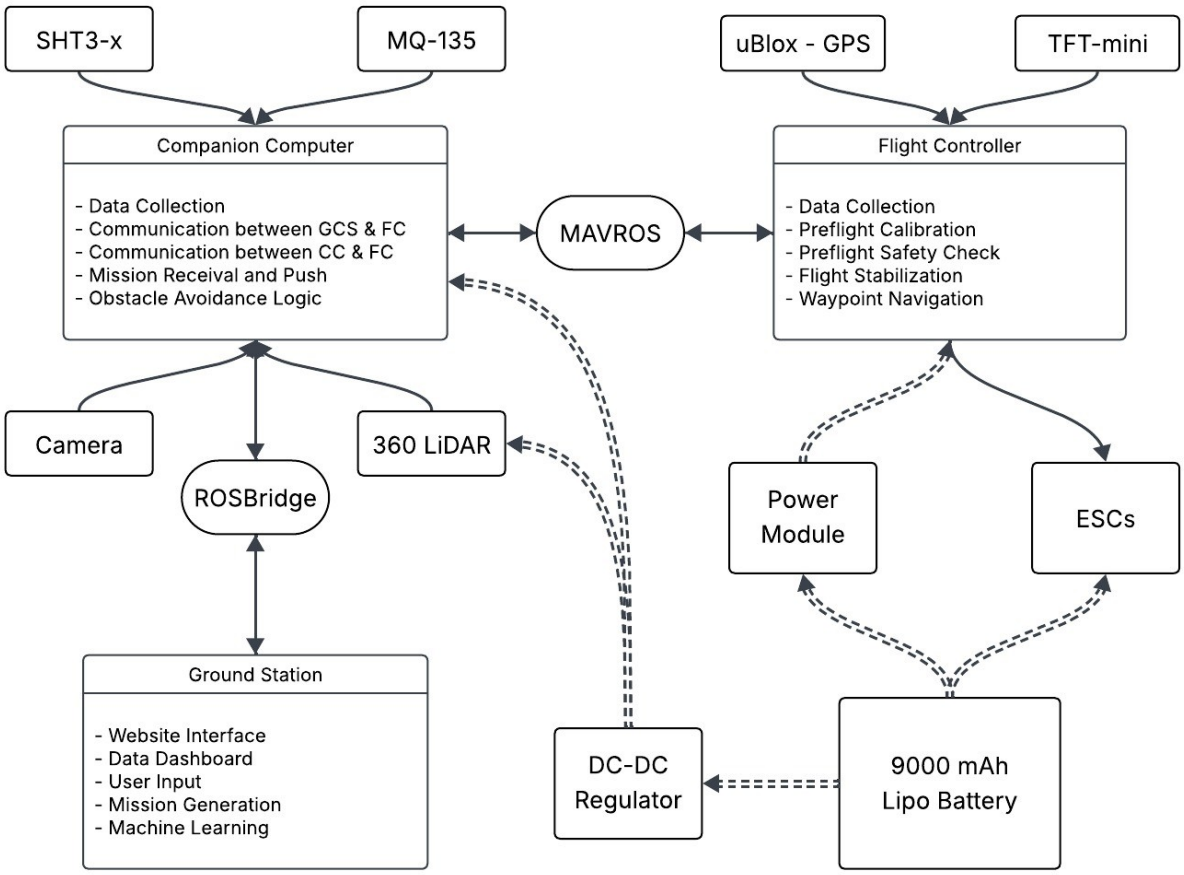

Figure 7 – Block diagram of the proposed system

1.7 Report Structure

This report will be divided into six chapters. Covering theoretical studies, hardware and software implementation, and design of the UAV system.

This first chapter has presented a brief overview of the project and its goal. This served as an introduction to the autonomous UAV fire detection that we're going to implement, along with the motivations that led to this work, and all the constraints and limitations the project can face.

Chapter 2 will show already existing systems, papers, theoretical studies, research accomplished, publications, and some real-world applications related to the proposed work.

The third chapter will present all hardware related to electronics and communication protocols engineering tools. In this part, all sensors and actuators selected will be showcased, and circuit designs.

Chapter 4 will present the software part of the system, the database management, and the Raspberry Pi codes using ROS and the flight controller firmware, along with ArduPilot Diagrams.

The fifth chapter will present all parts related to the assembly, testing, and validation of the system. This phase is interesting due to the complicated merging conflicts and synchronization problems occurred.

Finally, in chapter 6, conclusions and recommendations for improvements for future iterations of the design, the impact of the proposed solution on different constituents, as well as other future work that could complement this project, are discussed.

Chapter 2: State of the Art

This chapter presents a global review of research, technologies, and systems related to autonomous UAVs, with a focus on applications in search and rescue missions, fire detection, and victim localization. The purpose of this review is to establish a strong foundation in the field of autonomous UAVs by analyzing previous work, systems, theoretical studies, methodologies, and innovations. It also aims to provide an understanding of the current advancements in drone technology.

To achieve an efficient design and development of the autonomous UAV system, the studies and works are organized into several thematic areas aligned with our project's objectives. These include, first, the aerodynamic systems that constitute the drones, followed by a description of sensor integration, AI, and image processing for fire and victim detection. Furthermore, the chapter discusses autonomous navigation algorithms and obstacle avoidance to enable real-time decision-making, as well as the communication systems used by UAVs to connect with the ground station. Finally, several systems developed in this domain will be presented.

2.1 Aerodynamics of the UAV systems

Aerodynamics is a fundamental aspect in the implementation, performance, and design of UAVs, as it affects their stability, flight efficiency, and energy consumption. An effective aerodynamic design reduces air resistance (drag) and increases lift, allowing drones to consume less battery power during long flights, a key consideration in time-critical scenarios involving limited battery life (Dündar, 2020). For autonomous UAV systems used in dangerous missions, such as search and rescue, aerodynamics must be carefully optimized to ensure the safest and most reliable flight in hazardous environmental conditions.

Two of the most significant and common UAV configurations are:

- **Fixed-wing UAVs:** Characterized by their long endurance and ability to cover vast areas, they generally consume less energy thanks to their capacity to maintain lift with less continuous thrust. This makes them suitable for long-distance missions and environmental monitoring. However, their inability to hover and operate in confined

spaces limits their usefulness for assessment or rescue operations in compact environments (Mathew, 2024). Figure 8 shows a Fixed-wing UAV.

Figure 8 – Fixed wing UAV (Manuel, 2024)

- **Multirotor UAVs (quadcopters):** Featuring Vertical Take-Off and Landing (VTOL) capability, the ability to hover, and high maneuverability. The aerodynamic performance of these UAVs is primarily determined by the rotor blade designs, thrust-to-weight ratio, and frame structure (Huimin Zhu, 2021). The drag coefficient and rotor blade pitch significantly impact energy consumption during hovering and forward flight (Md. Shah Alam, 2020). These characteristics make multirotors, especially quadcopters, the preferred choice for search and rescue missions as well as fire detection. Figure 9 shows a Quadcopter UAV.

Figure 9 – Quadcopter UAV (Prindle, 2019)

2.1.1 Aerodynamic Forces

A quadcopter is governed by four fundamental forces: lift, thrust, drag, and weight. The resultant of these forces determines its movement, directly influencing flight performance, stability, and energy efficiency.

Lift and thrust are generated by the rapid rotation of multiple propellers, which accelerate airflow downwards, thereby creating an upward reactive force (Anderson, 2017). The lift must always exceed the UAV's weight to enable it to ascend, while thrust modulation allows the UAV to move both vertically and horizontally (Austin R. , 2010).

The drag force — the aerodynamic resistance caused by turbulence from the airframe and the disturbed airflow produced by spinning propellers — impacts power consumption and limits the maximum flight duration (Tobias Merz, 2021). Additionally, induced drag increases significantly with payload weight and during complex maneuvers. Figure 10 shows the Aerodynamic forces.

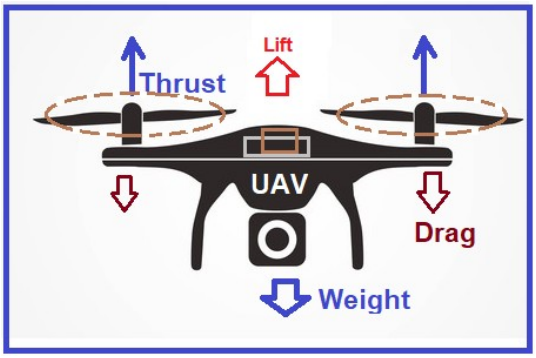

Figure 10 – Aerodynamic Forces (Pachpute, 2024)

2.1.2 Handling Disturbances

Aerodynamics can become highly complex due to the presence of turbulence, ground effect, and wind disturbances, all of which can hinder flight and create system instability in the air.

To better address these challenges, recent advancements include:

- **Computational Fluid Dynamics (CFD) simulations:** These simulations enable engineers to model airflow around the UAV structure, identify drag-inducing components, and estimate the impact of different frame geometries on lift generation

and airflow disruption. Studies have shown that frame symmetry and aerodynamic simplifications significantly improve flight stability under windy conditions (Wei Li, 2021).

- **Wind tunnel testing:** This method is used to optimize UAV frame and propeller design, allowing for the fine-tuning of structural and propeller configurations to achieve maximum aerodynamic efficiency.

Moreover, the flight control algorithm must account for aerodynamic disturbances to maintain stable flight. Autonomous UAVs typically rely on feedback from IMU sensors, gyroscopes, and barometers within the flight controller to dynamically adjust motor speeds and preserve equilibrium (Péter Megyesi, 2021). Advanced flight controllers implement adaptive Proportional-Integral-Derivative (PID) control systems that respond in real time to aerodynamic changes such as wind disturbances and payload weight variations (Megyesi, 2021).

2.1.3 Materials and Structural Considerations

The selection of materials and modular airframes in UAV design directly impacts aerodynamic performance. Reducing weight by using lightweight materials such as carbon fiber and ABS plastic is essential to improving thrust efficiency and extending flight times, without compromising structural strength (Hossam ElFaham, 2020).

Lightweight construction is particularly important for UAVs intended to carry thermal cameras, LiDAR systems, or communication modules. Since these payloads increase the overall mass, they require greater thrust output to maintain flight performance. excessive weight can lead to increased drag forces and power consumption, necessitating optimized aerodynamic shaping and propulsion system adjustments (Anderson J. , 2018)

Modular airframe designs enhance UAV flexibility by enabling quick changes of payload without major structural modifications (Valavanis & Vachtsevanos, 2015). However, excessive weight reduction or improper material selection can compromise structural rigidity and aero-elastic stability, leading to potential flutter or fatigue failures (Dewey, 2021). Figure 11 illustrates the UAV movement.

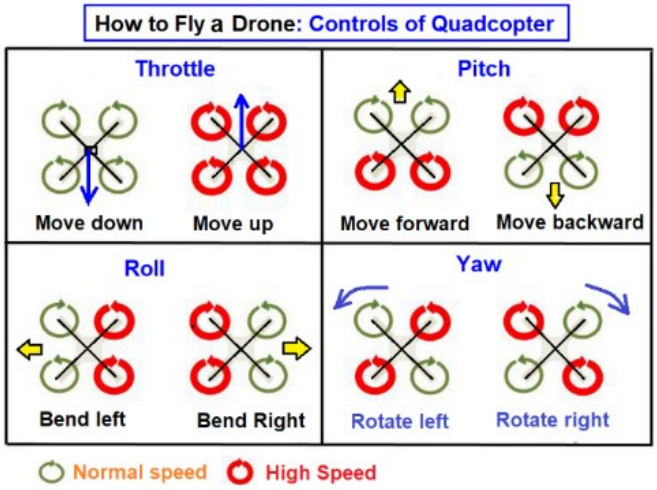

Figure 11 – Quadcopter movement (Pachpute, 2024)

To sum up, aerodynamic considerations are essential for maintaining drone performance and reliability, whether through rotor design, control algorithms, or frame structure. Optimizing aerodynamics is crucial for enhancing stability and flight duration in real-world applications. Well-balanced aerodynamics ensure efficient flight while performing multiple tasks.

2.2 Fire Detection and Monitoring Using UAVs

Fire detection using UAVs has become an essential operation in search and rescue missions, particularly in wildfire environments. UAVs offer a unique opportunity for early detection, rapid response, and continuous monitoring of fire outbreaks in areas that are difficult to access (Merino, 2012).

Traditional fire monitoring methods, such as ground sensors or satellite imaging, are often limited by environmental factors like smoke, weather conditions, and delays in data transmission (Luis Merino, 2012). By employing advanced sensors, UAVs have the capability to overcome these limitations by providing real-time, high-resolution imagery, thermal mapping, and automated analysis.

2.2.1 Imaging Fire Detection

Imaging-based fire detection is one of the most advanced technologies used in fire identification and management. Unlike traditional methods, it integrates advanced cameras to

capture heat signatures or fire-related phenomena, providing real-time data to enhance decision-making and response efforts. Two primary types of cameras are used in fire detection: thermal cameras and standard RGB cameras. These enable UAVs to operate effectively in challenging environments.

Thermal cameras: Thermal cameras detect infrared radiation emitted by heat sources, making them invaluable for fire detection since they can identify hot spots even through smoke or vegetation, where visible light cameras would fail (Lei Zhang, 2020). Studies have shown that this approach reduces the detection time of UAV systems by up to 40% compared to traditional monitoring methods. It can also help differentiate between fires and other heat sources, ensuring more accurate identification of fire boundaries (Jian Zhou, 2019). FLIR's Boson and Tau series are among the most commonly integrated thermal systems in UAVs for fire detection due to their high resolution, radiometric capabilities, and real-time thermal data provision for accurate fire mapping (Boson & Tau 2 Series, 2021).

RGB cameras: RGB cameras capture detailed, high-resolution images in the visible spectrum, providing visual context that complements thermal data. These cameras, combined with image processing techniques, enable fire detection and offer vital information regarding the fire's size, location, and potential impact (Adrian Luis Hernandez, 2021). Several AI algorithms can be applied to RGB fire detection systems to automatically identify fire-related features such as color, shape, and movement. For example, a study integrates Convolutional Neural Networks (CNNs) in fire detection, showing efficient results even in challenging conditions like smoke or variable lighting (Cheng Liu, 2021).

Combining RGB and thermal cameras enhances the UAVs' ability to provide both real-time visual context and heat detection, creating a powerful multispectral approach to fire detection. This dual-sensor capability ensures that UAVs can operate efficiently in complex fire environments (David Smith, 2023). Figure 13 shows different methods for fire detection.

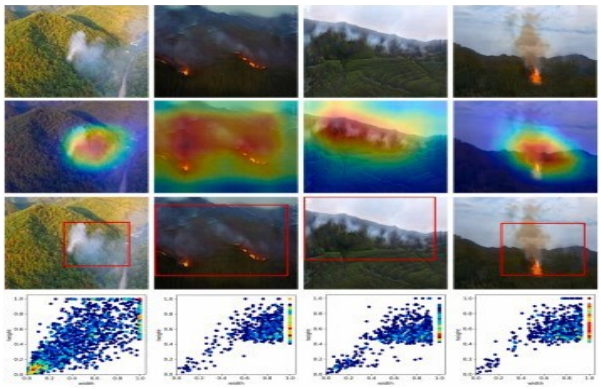

Figure 12 – Fire detection methods (Minsoo Park, 2022)

2.2.2 Multi-Sensor Fire Detection

Although cameras are efficient, combining multiple sensor types can further improve the efficiency and accuracy of fire detection operations by compensating for individual sensor limitations, increasing data reliability, and enabling earlier, more precise fire identification sensor fusion (thermal + gas + spectral) improves detection confidence from 72% (single sensor) to 92%. Multispectral sensors and gas sensors have been integrated with UAVs, especially for complex environments. For example, multispectral sensors capture data across multiple wavelengths, which can be used to detect changes in vegetation or smoke density indicative of fire activity. Integrating CO_2 or smoke sensors into UAVs also provides valuable information for detecting fire emissions (Yong Zhang, 2020)

2.2.3 Fire Spread Monitoring

In addition to fire detection, UAVs can be used for monitoring, predicting fire spread, and providing real-time data for decision-making. In large wildfires, knowing the direction and heat signature of fire spread is essential for improving firefighting and evacuation planning. UAVs equipped with thermal sensors, LiDAR, and high-resolution cameras can provide detailed maps of the fire's size, intensity, and surrounding terrain, aiding authorities in planning containment strategies. Typically, UAV systems utilize a Kalman Filter for tasks like state estimation and sensor fusion (Jun Yang, 2019).

The use of UAVs in dynamic fire spread modeling,with a system that combines thermal imaging with LiDAR data to provide precise, real-time fire spread predictions (Vivek Kumar H. L.,

2020). The UAV system can predict the direction of fire movement and assess the likelihood of its spread. Their study concludes that real-time data from UAVs significantly improves firefighting efficiency by reducing response time and resource consumption. Figure 14 illustrates an AI algorithm designed to predict fire spreading.

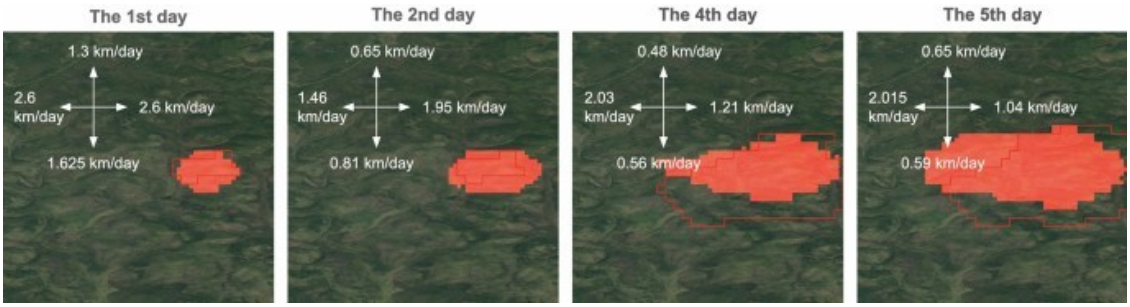

Figure 13 – Wildfire spreading predictions (Ivan Shadrin, 2024)

2.3 Victim Localization

During disasters, many victims remain unlocated, making victim detection and localization crucial for search and rescue (SAR) operations, particularly in scenarios such as earthquakes, snowstorms, or building collapses. UAVs offer a significant advantage in these scenarios by covering wide areas quickly and accessing locations that may be hazardous to human rescuers. The effectiveness of UAVs in SAR missions is further enhanced by integrating powerful detection algorithms and localization techniques that enable UAVs to recognize and pinpoint victims.

2.3.1 Victim Detection Methods

Thermal cameras: One of the most common technologies for detecting victims in SAR missions is thermal imaging. As mentioned earlier, thermal cameras detect body heat emitted by humans, which is particularly useful in environments where victims are not visible. For example, a UAVs equipped with thermal imaging can detect human body heat with 85% accuracy even in the presence of smoke during real-world disaster scenarios, making it an efficient tool for early victim detection (Vivek Kumar H. S., 2020). However, thermal imaging is limited by environmental factors such as intense fire, which can overwhelm human heat signatures, or snow. It also fails to detect deceased victims (Smith, 2023).

RGB cameras: RGB cameras can complement or even replace thermal imaging by providing high-resolution, color images that help identify victims and distinguish them from other heat sources. A research shows how ordinary cameras, combined with Machine Learning (ML), can automatically detect human forms (Syed Ahmed, 2021). For instance, the integration of Convolutional Neural Networks (CNN) to train UAV systems to detect fires using both thermal and RGB images, achieved a 40% reduction in detection times compared to older methods such as satellite monitoring (Wei Liang Zhang T. L., 2020). Similarly employing You Only Look Once (YOLO), an AI model into UAV systems, to automatically detect human figures in visual images showed high accuracy detection (Ahmed, 2021). Figure 15 illustrates human detection.

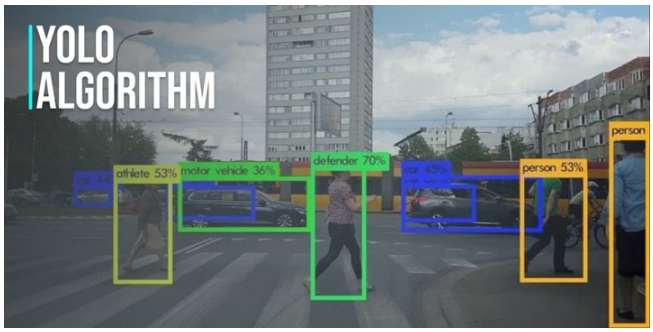

Figure 14 – Object detection (Anastasios Karagiannakos, 2019)

2.3.2 Victim Tracking

After victim detection, the next step is precise localization and tracking. Advanced UAV systems typically use GPS, LiDAR, and multisensory data to achieve accurate localization. (Zhang W. L. 2020) and (Wei Liang Zhang T. L., 2020) demonstrated how GPS and LiDAR sensors can generate a 3D map of the environment and localize victims and fires effectively.

Another important technique for victim localization and movement tracking in dynamic environments is Simultaneous Localization and Mapping (SLAM). SLAM algorithms have been widely adopted to enhance tracking precision. (Hong Zhen Li, 2021) showcased the use of SLAM for victim localization in dense urban environments, enabling UAVs to continuously map their surroundings and detect victims in real time.

2.4 Computer Vision in UAV Systems

2.4.1 Obstacle avoidance

In autonomous UAV missions, obstacle avoidance is an important task, especially in dangerous environments. It manages the UAV track and re-routes the path to avoid collision with obstacles that get in the way. Traditional navigation methods struggle with real-time response and environmental doubt. Thus, recent research has focused on combining artificial intelligence with different sensor systems like LiDAR, stereo vision, and ultrasonic systems.

LiDAR sensors emit a laser pulse and measure its return time, then generate dense provide high-resolution 3D point clouds, giving precise distance estimation and obstacle geometry mapping. In a study by (Ankit Kumar, 2021) an autonomous UAV equipped with LiDAR combined with a voxel-based occupancy grid map and path planning algorithm. This model enables the UAV to fly steadily while detecting and avoiding both static and dynamic obstacles in real time.

In addition, stereo vision systems, which use disparity maps from dual cameras, have been integrated with deep learning for path planning and depth estimation (Yue Zhou, 2020). (Ming Chen, 2021) has worked on a hybrid model combining stereo vision and inertial data, achieving accurate obstacle prediction in GPS-denied environments using a modified DeepLabv3+ segmentation network. UAVs have also been trained for autonomous avoidance behaviors using RL techniques, which allow them to learn by trial and error to adapt to new obstacles (Nguyen, 2021). *

2.4.2 Autonomous Navigation

AI integration is also widely used in autonomous navigation and path planning. UAVs employ path planning algorithms and Reinforcement Learning (RL) to navigate dangerous and unknown areas. Traditional navigation systems often suffer from signal degradation or loss due to occlusions or electromagnetic interference. (Hoang Nguyen, 2021) discusses AI models using RL for UAV navigation in fire zones. In this approach, the UAV uses feedback from sensors such as

LiDAR and IMUs to determine optimal routes through disaster areas. The UAV was able to avoid obstacles and find the most efficient paths to victims' locations.

Computer vision, integrated with AI algorithms, also plays an important role in obstacle avoidance and motion tracking. For instance, SLAM algorithm enables UAVs to build a map of unfamiliar environments while simultaneously determining their position within them (Li Zhou, 2020)). Li demonstrated the use of SLAM in UAV systems for indoor SAR missions. A more extensive overview of SLAM for UAV-based exploration in GPS-denied environments is provided by combining visual data from RGB cameras with depth information from LiDAR, UAVs can localize themselves and detect victims. Figure 16 illustrates path planning and obstacle avoidance using SLAM.

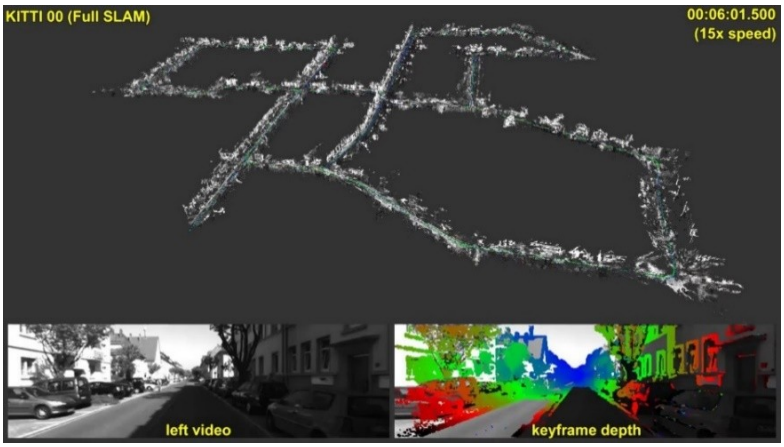

Figure 15 – Stereo Vision Mapping (Raul Mur-Artal, 2015)

2.5 Communication Systems in UAVs

Successful autonomous UAV operations typically rely on robust communication systems, especially for complex and hazardous tasks such as search and rescue (SAR) and fire detection missions. To ensure that operators continuously receive updates, control inputs, and video feeds, communication technologies facilitate real-time data transfer between drones and ground control stations. When UAVs operate in remote or GPS-denied locations, these systems also enable remote control and data relay. Several communication technologies are commonly employed in UAV systems, each with distinct capabilities and limitations.

For long-range communication, suitable for regional environments or fire detection operations, 4G and 5G networks offer high-speed data transfer with low latency, making them ideal for real-time video streaming. (Li, 2020)demonstrated that these networks provide faster and more reliable communication for autonomous UAVs.

Additionally, Radio Frequency (RF) communication is frequently used in UAV systems, particularly for low-data-rate applications like sending simple telemetry data. RF systems are effective and widely adopted, though they can be susceptible to interference.

Finally, in GPS-denied areas where standard communication systems like Wi-Fi or mobile networks are unavailable, satellite communication provides extensive coverage. SATCOM systems are especially valuable for long-range UAV operations in disaster environments, such as remote wildfires or ocean and snow rescues. (Huang, 2021) demonstrated how Very Small Aperture Terminal (VSAT) satellite communication links provide global connectivity to UAVs operating in rural and isolated regions.

2.6 Realized systems

Several realized UAV systems are widely used in SAR, particularly for fire and victim detection, with specialized equipment to enhance operational capabilities:

AeroVironment Raven: A military search and rescue VTOL fixed-wing UAV with a 90-minute flight endurance. It is equipped with IR and LiDAR sensors and carries the advanced Mantis i45 EO/IR gimbal for dual thermal (640×512) and visible-spectrum imaging, capable of detecting heat signatures from up to 1,200 meters. It features an automated grid-pattern survey and onboard AI edge analytics for real-time hotspot detection. The Raven is resilient to 25 mph winds and light rain and offers a 1.2 kg payload capacity suitable for sensors like gas analyzers. (AeroVironment, 2020). Figure 16 illustrates the AeroVironment Raven UAV.

Figure 16 – AeroVironment raven UAV (avinc, 2020)

DJI Matrice 300 RTK: A quadcopter UAV with IP45 weatherproofing designed for complex environments with extreme temperatures ranging from -20°C to 50°C. It has a 55-minute flight time supported by dual-battery hot-swap technology. The UAV integrates a 20MP zoom camera, a 640×512 radiometric thermal sensor, and a 1,200 m laser rangefinder for precision imaging. It also includes AI capabilities for real-time human identification in obscured environments and autonomous BVLOS (Beyond Visual Line of Sight) inspection routes. For safety, the UAV is equipped with 360º omnidirectional obstacle sensing. (DJI, 2021). Figure 17 illustrates the DJI Matrice 300 RTK UAV.

Figure 17 – DJI Matrice 300 RTK UAV (Edinburghdronecompany, 2023)

AUTEL Robotics EVO II Dual: This foldable quadcopter combines portability and advanced sensing, delivering 40 minutes of endurance with its 6,000mAh battery and 360° obstacle avoidance through 12 vision sensors. Its dual-sensor payload features an 8K 48MP RGB camera and a 640×512 thermal camera. AI-driven Smart Track recognizes humans and vehicles, while real-time 3D mapping supports fire spread prediction. Operational robustness includes 5G connectivity with 20ms latency and reliable performance in environments as cold as -10°C (Robotics, 2020). Figure 18 illustrates the AUTEL Robotics EVO II UAV

Figure 18 – AUTEL Robotics EVO II Dual UAV (autelrobotics, 2025)

Although lots of systems have been realized, the main contributions and differences between the proposed system and existing ones are the following:

Options	AeroVironment Raven	DJI Matrice 300 RTK	EVO II	The proposed system

2.7 Conclusion

In summary, this chapter presents a comprehensive review of autonomous UAV systems, focusing on their roles in fire detection and victim localization. It begins by exploring the aerodynamics and mechanics of UAV systems, especially multirotor UAVs and flight controllers, highlighting their capabilities to maintain stability, efficient flight time, hovering, and maneuverability—essential qualities for search and rescue (SAR) missions.

The chapter also provides an overview of advanced technological equipment used in SAR operations and their applications in fire detection and victim localization. This includes the use of multi-sensor systems, cameras, and AI algorithms. The Combination of sensors and AI image processing has improved SAR operations, making it more efficient, precise, and even faster and reliable. However, many challenges and limitations face these improvements, such as flight time, environmental conditions, and data processing

AI algorithms and computer vision play a central role in the studies, particularly in obstacle avoidance and navigation algorithms, which are crucial for effective SAR missions. Additionally, various communication systems are discussed for improving data transmission between drones and ground stations, ensuring the fastest possible delivery of critical information to save time.

Finally, several accomplished UAV systems currently used in real-world rescue operations are examined, with an emphasis on their functionalities and integrated equipment.

Chapter 3: Hardware Implementation

After presenting an introduction and literature analysis of the autonomous UAV project, a demonstration of the hardware system chosen will follow. Processing units, sensor integration, and actuators that will allow the UAV to function autonomously during missions will all be covered in this chapter. By bridging the gap between theoretical research and practical applications, the hardware implementation turns the suggested design into a reliable, workable system with the ability of intelligent decision-making, real-time perception with high stability.

The chapter starts by introducing the overall system overview, giving a detailed explanation of the system block diagram, which shows the interconnection between subsystems and the general knowledge of the components, then a detailed explanation of each component describing their specific performance criteria, such as reliability, weight, circuit wiring, and computational efficiency to ensure the best performance and stability of the system for disaster reconnaissance and SAR missions under challenging conditions. Furthermore, we will discuss power consumption, ensuring energy management and power delivery for safe and endurance missions.

This chapter sets the basis for the software system presented in the following chapter by ensuring the best selection of the hardware elements that support the autonomous flight and the critical task

3.1 System Overview

The system hardware components are structured following the general architecture already presented in Figure 7 (Section 1.6), which outlines the UAV platform, ground station, and communication link.

The autonomous UAV system relies on many integrated hardware components that generally enable navigation, sensing, collecting data, and communication. Figure 17 showing the block diagram presents the high-level architecture system.

In this block diagram, as mentioned in Chapter 1, the system is decomposed into two sub-systems: The drone and the ground station.

Starting with the drone, a Pixhawk 2.4.8 serves as the flight controller, the core of UAV navigation and control. It interfaces with four ESC (electronic speed control) units that receive a PWM signal from the Pixhawk, which deliver the specific speed and power to four brushless DC motors. In addition, a GPS (Global Positioning System) module and a compass provide position and orientation data to the Pixhawk for waypoint tracking, navigation, and a stable flight. A LIDAR for safe landing, all connected to the Pixhawk through serial communication.

A Raspberry Pi 4 B serves as a companion computer to collect and process real-time data from the sensors, including a camera for live video streaming, an MQ-135 air quality sensor for measuring the CO2 gas levels, a SHT3X for temperature and humidity, and a 360 LIDAR X4 PRO for obstacle avoidance. Sensor data is sent to the ground station via a GSM module. The Mission File and obstacle avoidance commands will be transmitted by the companion computer to the flight controller via serial communication.

The power unit consists of three Lipo batteries rated at 3S with a capacity of 3000 mAh each, connected in parallel for a 9000 mAh total. It is connected to a power module that powers both the Pixhawk and the ESCs through a power distribution board. Additionally, a 5A XL4015 DC to DC step-down regulator is connected to the batteries to power the Raspberry Pi, sensors, and the 360 LIDAR.

Moving on to the ground control station, a computer server receives all data from the drone: displays live video and sensor data as well as monitors the flight. Moreover, it generates mission files based on user input and enables control, as well as detecting fire and victims using machine learning algorithms discussed further in the upcoming chapters.

3.2 Components

This section provides a detailed description of the hardware components integrated in the autonomous UAV system. The components are selected based on their capability and functionality in the system and the performance requirement that leads to a successful system. The components are grouped based on their role in the system.

3.2.1 Processing units

The processing units are the brain of the system; they are responsible for controlling the UAV flight, giving orders, and handling onboard data processing. That includes the Pixhawk 2.4.8 flight controller that is responsible for flight aerodynamic stability, and the companion computer, Raspberry Pi 4 B, for sensor data processing and running high-level algorithms.

3.2.1.1 Pixhawk 2.4.8

As previously mentioned, the Pixhawk 2.4.8 is the flight controller used in the system. It handles low-level control tasks, including navigation and stabilization, and receives orientation data from onboard sensors such as the GPS module and the compass. It processes this data and outputs PWM signals to the ESCs.

The Pixhawk 2.4.8 is powered by a 32-bit ARM Cortex M4 microcontroller with a Floating-Point Unit (FPU) running at 168 MHz, supported by 256 KB of RAM and 2 MB of flash memory. It also includes a 32-bit failsafe co-processor. The built-in sensor suite comprises an MPU 6000 as the main accelerometer and gyroscope, an ST Micro 16-bit gyroscope, an ST Micro 14-bit accelerometer/compass (magnetometer), and a MEAS barometer.

Regarding connectivity, Pixhawk offers 5 UART serial ports, as well as I2C, SPI, USB, and CAN ports. These interfaces facilitate communication with various sensors and components, including the GPS, compass, LIDAR, and Raspberry Pi. Additionally, the Pixhawk provides a PPM input for the radio control receiver, an S. Bus output, and a total of 8 main plus 6 auxiliary PWM outputs to control the ESCs (Ardupilot, 2024).

Pixhawk 2.4.8 was selected for its capability to support advanced autonomous features and for its open-source architecture. It runs on ArduPilot firmware, which enables sensor calibration,

parameter tuning for flight control, navigation management, stability control, and many other functionalities that will be elaborated on in the software section. Moreover, the Pixhawk 2.4.8 can interface with a companion computer via the MAVROS protocol over MAVLink, allowing the UAV to respond to data processed by the secondary processing unit.

Compared to other flight controllers like SpeedyBee or APM, the Pixhawk offers greater flexibility for integrating multiple sensors, seamless compatibility with the Robot Operating System (ROS) via MAVROS, and comprehensive mission planning capabilities. Figure 1illustrates the Pixhawk 2.4.8 and its interface.

Figure 19 – PIXHAWK 2.4.8 flight controller (Ardupilot, 2024)

3.2.1.2 Raspberry Pi 4 B

The Raspberry Pi 4 B serves as the companion computer responsible for collecting and processing real-time data from sensors, transmitting it to the ground station, and communicating with the flight controller to manage autonomous flight based on specific sensor inputs such as LIDAR. The Raspberry Pi 4 B is equipped with a 64-bit quad-core Cortex-A72 microprocessor running at 1.5 GHz, 8 GB of RAM, and 64 GB of storage via an SD card. It also features integrated Bluetooth and Wi-Fi modules.

Programming the Raspberry Pi can be done directly through terminal access using its HDMI and USB ports to connect a monitor, mouse, and keyboard. Alternatively, remote access is possible via the SSH protocol. The Raspberry Pi offers multiple interface options, including two USB 2.0 ports used for the camera and LIDAR data, and a 40-pin GPIO header dedicated to serial communication interfaces such as I2C, UART, and SPI, which are used for various sensors. The

GPIO also provides 3.3V and 5V power outputs along with ground pins. It operates at 5V, supplied by the DC-DC regulator, with a peak current draw of 3A under full load.

Regarding the operating system, the Raspberry Pi runs on Ubuntu OS. Compared to other microcontrollers, the Raspberry Pi is the most suitable companion for this project because it supports the Robot Operating System (ROS), which facilitates communication between sensors and actuators. This makes system programming straightforward, a topic that will be discussed in detail in the next chapter.

Additionally, the Raspberry Pi offers higher computational power than microcontrollers such as the Arduino or ESP32, while remaining less expensive than microcomputers like the Jetson Nano, making it an ideal choice for this project. Figure 19 illustrates the Raspberry Pi 4 B and its pinouts.

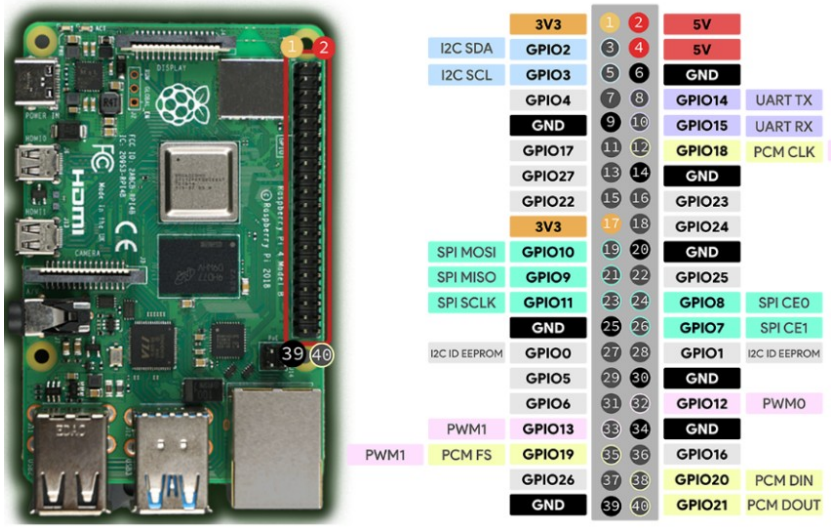

Figure 20 – Raspberry Pi 4 Model B (Santos, 2023)

3.2.2 Actuators

The actuators are responsible for executing the control commands from the Pixhawk flight controller to perform the autonomous flight. It includes the electronic speed controllers (ESC) and the Brushless DC motors, which operate together to produce thrust, stability, and control direction.

3.2.2.1 Electronic speed controller

The Electronic Speed Controller (ESC) is an integrated electronic circuit that uses electronic devices a microcontroller, a MOSFET, a capacitor, and a PWM signal processor, to control the direction, the speed, and the break of a brushless DC motor. ESC receives a PWM signal from Pixhawk and transform it into a specific power level to drive the motors. ESC acts like a translator between the flight controller and the motors, ensuring the required speed adjustment for stable flight. Hobbyking ESC is used with 30 A each, which is suitable for the motor of the project. It is connected to the Pixhawk by a three-pin servo connector, delivering the signal with additional ground and an unpowered 5V line for standard interfacing (Hobbyking, 2017).

The ESC is powered by a 3-cell LIPO battery with 11.1 V through a powered distribution board, and it's connected to the brushless motors through three output wires (A, B, C) which deliver three-phase power required to generate a rotating magnetic field to spin the motors. Hobbyking ESC was selected due to BLHeli-S customization, allowing optimization for tailored braking behavior and throttle response, including robust protection features that will prevent damage during typical UAV missions, and it is suitable for small to medium drones without adding unnecessary bulk. Figures 21 and 22 show the ESC and the wiring to the power distribution board.

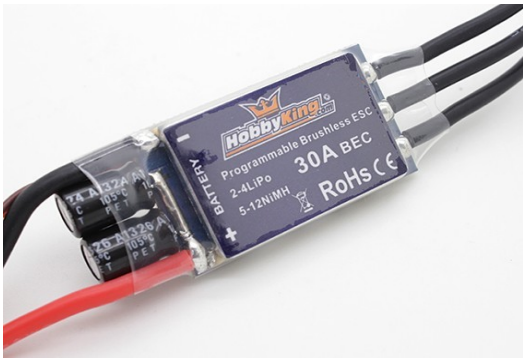

Figure 21 – ESC 30A (Hobbyking, 2017)

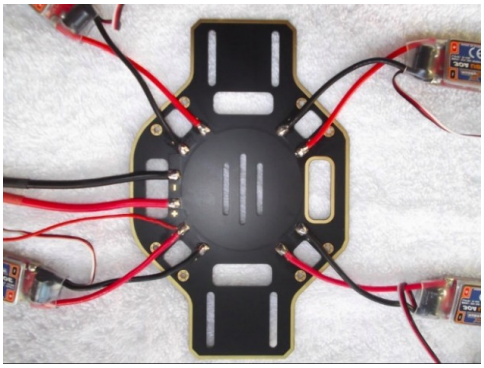

Figure 22 – ESC wiring (Vrinda, 2019)

3.2.2.2 Motors

The brushless DC motors produce the required mechanical thrust to lift, maneuver and stabilize. They convert the electrical energy controlled by ESC into rotating motion and in turn thrust, allowing the drone to take off, navigate, hover, and respond to the flight controller in real-time.

As we mentioned earlier, brushless motors work with three-phase power, emitting an electromagnetic field to rotate the rotor. In this project, we used the A2212/6T brushless DC motors with shaft diameter 3.17 mm operate at a voltage of 11.1 V with maximum power 342W and 2200 KV out-runner type designed for high-speed rotation and heavyweight UAV operations and it indicates high rotational speed per volt, so in our case ideally the motor will spin around 24000 RPM with no load (BeirutElectroCity, 2025).

Each one of the 4 motors is placed at the motor mount of the drone, equipped with the appropriate 10*4.5 propeller. The propellers are set with their lower hub aligned with the motor shaft and secured using a nut on the upper hub in a specific direction as shown in the Figure 23 where the leading edge (the raised part of the propeller) must be aligned with the motor direction so the drone can lift off. Then, the motor spin direction is set based on software spin or ESC wiring, as swapping two wires reverses the spin direction of the rotor.

Each propeller is capable of delivering up to 1000 g of thrust on full throttle. The 2200 KV brushless motor provides a high thrust-to-weight ratio, ensuring stable lift for the UAV. It is compatible with the 30 A ESC. Figures 23 and 24 show the Brushless DC motor and the 10-4.5 propellers.

Figure 23 – Brushless DC motor (BeirutElectroCity, 2025)

Figure 24 – 10*4.5propeller (Amazon, 2025)

3.2.3 Sensors

In this section, we demonstrate the sensors used in the UAV system that collect real-time positional, environmental, and localization data and send it to the processing units. It will include their role, the type of data, and how they are connected to each of the Pixhawk or Raspberry Pi

3.2.3.1 GPS and Compass Module

The Global Position System (GPS) and compass module are essential for autonomous. The GPS provides real-time location data, enabling the Pixhawk flight controller to determine the drone's geographic position and execute waypoint navigation, allowing the UAV to autonomously follow pre-defined mission paths.

In this project, the NEO-6M GPS module is used due to its fast position acquisition, high sensitivity, cost-effectiveness, and compatibility with the Pixhawk. The GPS module is connected to the Pixhawk via the UART interface on the Telemetry 3 port. The NEO-6M provides a horizontal position accuracy of 2.5 meters, a 1 Hz navigation rate, a navigation sensitivity of -161

dBm, and operates at a baud rate of 9600. The module functions within a temperature range of -40ºC to 83ºC, with a supply voltage of 3.3V and a current consumption of 45 mA.

Integrated within the same module is a 3-axis digital compass (magnetometer) based on the HMC5883L chip, which supplies directional orientation data to Pixhawk. The compass provides heading information, enabling the UAV to determine its orientation relative to magnetic north. In conjunction with the GPS, it ensures stable and accurate flight during autonomous operations. The compass is connected to the I2C port of the Pixhawk.

To minimize electromagnetic interference and ensure faster satellite acquisition, the GPS and compass module should be installed at the highest point of the UAV structure. Figure 25 displays the GPS and compass module.

Figure 25 – GPS and compass module (BeirutElectroCity, NEO-6M GPS, 2025)

3.2.3.2 Landing LIDAR

A safe landing for any UAV is a critical point, ensuring the UAV does not crash while landing when the mission is over, a mini LIDAR (Light Detection and Ranging) sensor can provide precise distance measurement from the drone to the ground, allowing the Pixhawk to adjust the throttle smoothly during the descent phase, preventing a hard landing.

The TF-Mini LiDAR module is used due to its high accuracy for short-range detection. It is a single-point micro ranging module that integrates a unique optical and electrical design based on the TOF (Time of Flight) principle. LIDAR emits and receives laser light; distance can be measured by calculating the time it takes for the reflected light to return.

The LIDAR is connected to the Pixhawk telemetry port using UART protocol and operates at a 5V supply voltage with 100Hz frequency from 0.3 to 12 meters, a temperature range between 20º and 60º, a baud rate of 115200, and 70,000lux light sensitivity with 1% accuracy less than 6 m. Figure 26 shows the TF mini LIDAR sensor.

Figure 26 – TF-mini LIDAR (RobotPiShop, 2025)

3.2.3.3 Air Quality

The MQ-135 is an air quality sensor integrated into the UAV system to detect the concentration of harmful gases such as CO2, particularly in wildfire environments. This enhances fire assessment capabilities and supports search and rescue missions by providing valuable air quality information.

It produces an analog output voltage ranging from 0 to 5V, proportional to the concentration of gases in the surrounding environment. It features a built-in heater and a sensitive layer that responds to changes in gas concentrations. However, since the Raspberry Pi is equipped only with digital GPIO pins, the MQ-135 cannot be connected directly. To address this, an MCP3008 Analog-to-Digital Converter (ADC) is used to convert the analog signal into a digital format compatible with the Raspberry Pi.

It is connected to the Raspberry Pi via the SPI interface, as illustrated in Figure 27. The MQ-135 is connected to channel 0 of the MCP3008, with a 5V supply provided by the Raspberry Pi. The MCP3008 converts the analog signal from the MQ-135 into a digital input that the Raspberry Pi can process.

The MQ-135 was chosen for its low cost, high sensitivity, and broad gas detection range. Figures 27 and 28 show the MQ-135 sensor and the connection setup between the MCP3008 and the Raspberry Pi.

Figure 27 – MQ-135 air quality (Robotpishop, 2025)

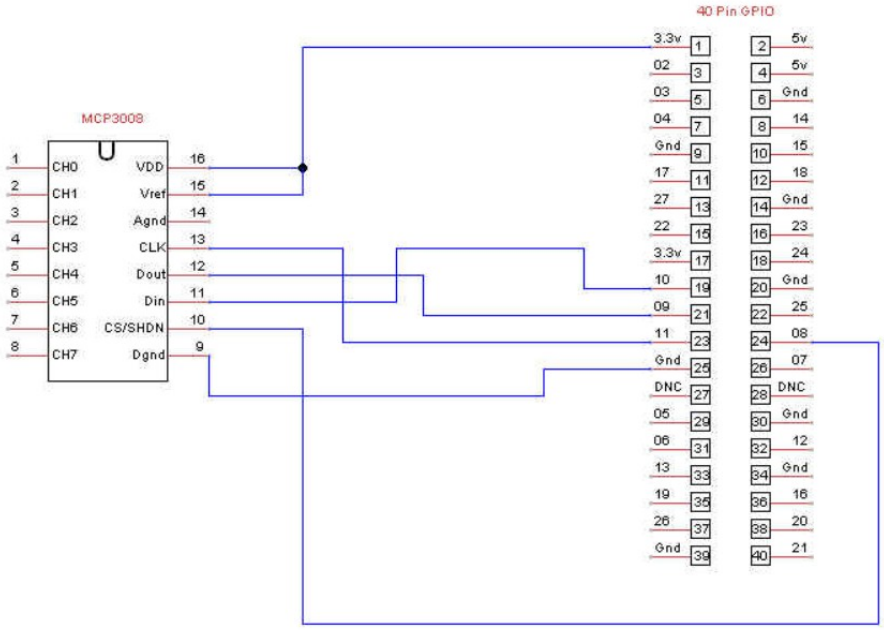

Figure 28 – MCP3008 circuit wiring (Raspberrypi, 2017)

3.2.3.4 Temperature and humidity

Temperature and humidity data are critically important in fire detection, as they help assess environmental conditions and enable fire spread prediction. The SHT3X is a compact digital sensor that reliably measures both temperature and humidity with high precision. It integrates a bandgap temperature sensor and a capacitive humidity sensor on a single CMOS chip, which provides a fully calibrated digital output.

The SHT3X is connected to the Raspberry Pi via the I2C interface, as illustrated in Figure 36. The I2C protocol enables efficient two-wire communication with a speed of up to 1 MHz, and

the sensor operates at a 3.3V supply voltage. The real-time data collected by SHT3X is processed by Raspberry Pi and subsequently transmitted to the ground station for further analysis and decision-making.

The SHT3X sensor offers excellent measurement accuracy, with ±2% for relative humidity and ±0.3°C for temperature. It supports a wide operating temperature range from -40°C to +125°C and can measure humidity across the full range of 0 to 100%. Figures 29 and 30 show the SHT3X sensor and its connection to the Raspberry Pi (RobotPishop, 2025), (RaspberryPi, 2024).

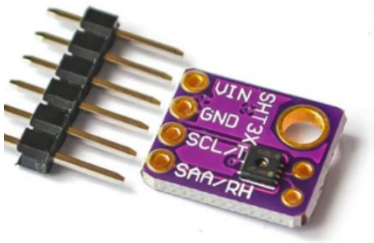

Figure 29 – SHT3X sensor (RobotPishop, 2025)

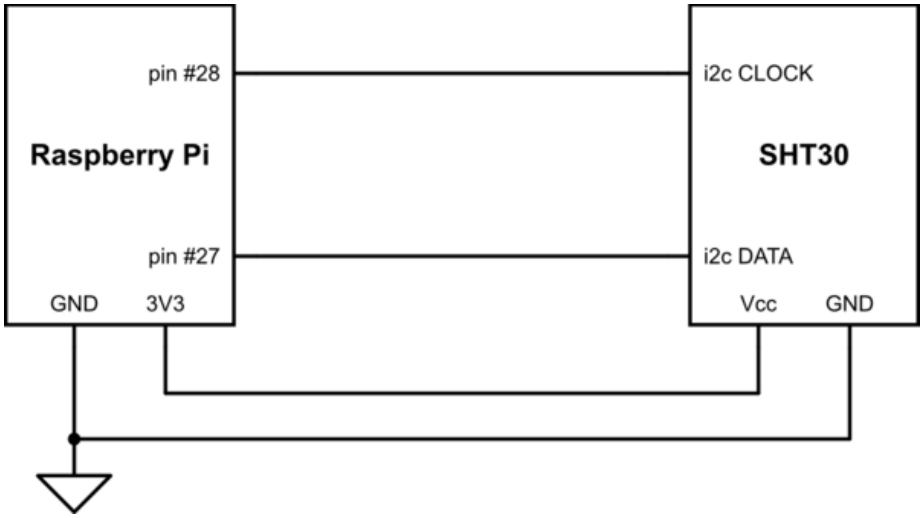

Figure 30 – SHT3X circuit wiring (RaspberryPi, 2024)

3.2.3.5 Camera

The camera plays a crucial role in SAR missions by supporting visual navigation and enabling live video streaming from the UAV system. Video data is transmitted by the Raspberry Pi

via the ROS bridge to the ground station over GSM. Using the camera feed in combination with machine learning algorithms, the ground station can detect fires and locate victims.

For this project, a 720p webcam with a frame rate of 30 fps was used, connected to the Raspberry Pi via USB. The webcam offers low latency and rapid video streaming, making it well-suited for SAR missions that demand quick response times. It is compatible with the Raspberry Pi, lightweight, and consumes low power. Figure 31 shows the webcam.

Figure 31 – Webcam (Astrum, 2023)

3.2.3.6 360º LIDAR

In autonomous UAV missions, LIDAR enables true autonomous navigation by allowing the drone to detect and avoid collisions with obstacles. It also assists in environmental mapping and prevents crashes in dynamic or cluttered areas. The LIDAR used in this project is the 360º YD LIDAR X4 Pro.

This sensor performs 2D distance measurements with 360º scanning based on the principle of triangulation. It emits a laser beam toward the surface; the reflected laser light returns to the sensor at a known angle relative to the emitter. The distance is then calculated using trigonometric relations. The LIDAR features a brushless DC motor for rotation and employs algorithms designed to achieve high-frequency, high-precision, and stable distance measurements with strong resistance to ambient light interference.

The YD LIDAR X4 Pro has a measurement range of at least 10 meters, a scanning frequency between 6 and 12 Hz, and a sampling rate of up to 5000 Hz. It supports the Robot Operating System (ROS), enabling the Raspberry Pi to process LIDAR data and send obstacle avoidance commands to Pixhawk via the MAVROS protocol using this project’s native avoidance algorithm.

Connectivity is established through a USB-to-UART adapter between the LIDAR and the Raspberry Pi for data transmission, while power is supplied from the DC-DC regulator at 5V with a maximum current draw of up to 1A during operation.

The YD LIDAR X4 Pro was selected for its high accuracy, long range, ROS compatibility, and compact size suitable for UAV integration. Figure 32 shows the 360º YD LIDAR X4 Pro.

Figure 32 – YD LIDAR X4 PRO (RobotPi, 2025)

3.2.4 F450 Frame

The F450 frame serves as the structural foundation of the UAV system, supporting all components. With a wheelbase of 450 mm, the F450 offers sufficient space and load capacity to accommodate the essential UAV components and payloads. Constructed from nylon-reinforced fiberglass, the frame combines a lightweight design—approximately 300 g—with durability to withstand impacts during crashes.

Its X-shaped design provides excellent balance and stability throughout flight missions. Pre-drilled mounting holes facilitate secure attachment of motors, the flight controller, and other hardware. The frame includes two stands designed to hold the processing units. Its layout ensures effective weight distribution and vibration damping, which enhances flight performance and improves data accuracy.

Additionally, the F450 features raised landing legs that protect the electronic components during landing impacts. The frame was selected due to its wide availability, low cost, compatibility with standard UAV parts, and suitability for civil defense and environmental missions that demand modular and scalable configurations. Figure 33 illustrates the F450 frame used in the UAV system.

Figure 33 – F450 drone frame (RobotPiShop, f450, 2025)

3.2.5 Power supply system

The UAV's power supply system provides stable and sufficient electrical energy to all components, including processing units, sensors, and actuators. Reliable and precise power distribution is essential to ensure consistent performance, safe operations, and stable autonomous functionality. The system consists of a LiPo battery as the main power source, a power module to regulate voltage for the Pixhawk, and a DC-DC regulator to supply appropriate voltage to the Raspberry Pi and the YD LIDAR.

3.2.5.1 LIPO Battery

The UAV is powered by three 3S (3-cell) lithium polymer (LiPo) batteries connected in parallel, each rated at 11.1 V, 3000 mAh, and a high discharge rate of 30C, which is critical for delivering the high current demands of UAV operations. Each battery is equipped with a T-plug for power output and a JST-XH connector for charging.

LiPo batteries are widely favored in UAV applications due to their lightweight design, high energy density, and ability to provide the necessary current for actuators and electronics. The selected batteries provide sufficient capacity for extended flight duration and are mounted on the

lower part of the frame to maintain balanced weight distribution and the correct center of gravity, ensuring system stability. The LiPo battery pack connects to the power module, which outputs a regulated 5.1 V supply to the Pixhawk and 11.1 V to the power distribution board. The power distribution board, in turn, supplies the ESCs and the DC-DC regulator. Figure 34 shows the LiPo batteries used in the system.

Figure 34 – 3S LIPO battery (electroSLAB, 2025)

3.2.5.2 Power Module

Power module in an electronic circuit regulates a specific voltage and monitors current and battery capacity. It receives 11.1V from the LIPO batteries and delivers 5.1V to the Pixhawk through a 6-pin connector, ensuring the required voltage for a safe flight controller performance and safe battery management. The power module also delivers 11.1V from the batteries to the ESC and DC-DC regulator through the power distribution board. Figure 35 shows the power module.

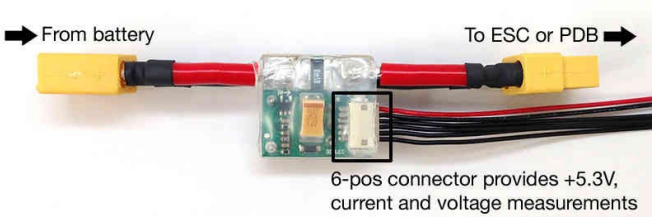

Figure 35 – Power Module (Ardupilot, Power Module, 2024)

3.2.5.3 DC-DC Regulator

The 5A XL4015 DC-DC regulator is used to convert the battery output voltage of 11.1 V from the power distribution board into a stable 5.1 V supply to power the Raspberry Pi and the YD LIDAR X4 Pro. This regulator accepts an input voltage range of 4 to 38 V and can output voltages from

1.25 V up to 36 V with a maximum current of 5 A and a maximum power of 75 W. It features a voltage error of ±0.05 V and includes a USB port.

The Raspberry Pi is connected to the 5 V output and ground pins of the regulator. The YD LIDAR is powered via the USB port of the regulator, connected through the LIDAR's adapter. Figure 36 shows the DC-DC XL4015 regulator.

Figure 36 – DC-DC regulator (ElectroSLab, 2025)

3.3 Power consumption

Ensuring stable and safe power distribution in the UAV system requires a detailed understanding of power consumption. This section analyzes how power from the source is distributed to optimize overall component performance, estimate voltage and current requirements, calculate system weight, and evaluate how these factors affect flight time. A precise power analysis prevents overloads and ensures the UAV receives adequate energy for optimal performance and endurance.

The UAV is powered by three LiPo 3S batteries (each with 3000 mAh capacity and 11.1 V nominal voltage) connected in parallel. This configuration maintains the voltage at 11.1 V while summing the capacities:

Total capacity = 3000 + 3000 + 3000 = 9000 mAh

Total energy E=11.1×9=99.9 Wh

However, considering safety margins, only 80% of the total energy is usable: Usable Energy=99.9×0.8=79.92 Wh

The battery voltage ranges through three states:

Fully charged: 12.6 V

Nominal (optimal): 11.1 V

Discharged (cutoff): 10 V

Actuator Power Consumption

The brushless DC motors are rated at 2200 KV, meaning they spin at 2200 RPM per volt.

At nominal voltage (11.1 V):

$$2200 \times 11.1 = 24{,}420 \text{ RPM}$$

At full charge (12.6 V):

$$2200 \times 12.6 = 27{,}720 \text{ RPM}$$

Current draw varies by load:

No load per motor: 0.5 A, total 0.5×4=2 A

Normal flight (moderate load): 4 to 10 A per motor, total 16 to 40 A

Heavy load: 13 A per motor, total 52 A

Table 1 shows the power consumption of the UAV in Nominal and Full charge.

Table 1 - Power consumption

	Voltage	Normal load	Total normal load	Heavy load	Total heavy load
		4 to 10 A	16 To 40 A	13A	52A
Nominal charge	11.1V	44.4 to 111.1 W	177.6 to 444 W	144.3 W	577.2 W
Full charge	12.6V	50 to 126 W	201.6 to 504 W	163 W	655.2 W

After demonstrating the power delivered by the batteries and the actuator power consumption, the next section will show the power needed by the processing units and sensors, and calculate the total power consumption. According to the components described before, a table shows the voltage, power, and current of every component.

Table 2 shows the current, voltage, and power of every component of the UAV.

Table 2 - Components power

Components	**Power (W)**	**Voltage (V)**	**Current I=P/V(A)**
Raspberry Pi 4	5	5	1
Pixhawk2.4.8	2	5	0.4
YD LIDAR X4 PRO	2	5	0.4
Camera	1.5	5	0.3
TF-mini LIDAR	0.3	5	0.06
GPS	0.15	5	0.03
MQ-135	0.9	3.3	0.273
SHT3X	0.015	3.3	0.0045
Total power	11.87 W		

Total current from 5V supply ==2.19 A

Total current from 3.3Vsupply==0.277A

Total current $I_{3.3}+I_5$ =2.19+0.277=2.467A

Weight calculation is very important, ensuring drone stability and motor capability of lifting the drone addition to the flight time, depending on the load

Table 3 shows the cost and weight of every component of the UAV.

Table 3 - Cost and Weight

Components	Raspberry Pi 4	Pixhawk	YD LIDAR	Camera	TF-mini	GPS

					LIDAR	
Weight (g)	47	40	180	30	5	15
Cost($)	140	120	90	15	40	10

Components	MQ-135	SHT3X	F450 frame	Motors(4)	ESC(4)	Propeller(4)	Batteries(3)
Weight (g)	12	10	300	200	128	56	300
Cost(g)	3	3	34	40	20	6	48

The total component weight is 1350g, with around 50 g of wire; the total weight of the system is 1400 g. Total cost $569.

Total flight power is equal to the total power of the motor + the total power of the components for a full load =655+11.87 =666.87W, and for a cruise power =266+11.87=278.2W

The flight duration is equal to the energy used over the total power =80%of the total energy over the total power for a

full throttle of 79.92/666.87=7.18 minutes, and for a moderate throttle of 79.9/266= 18 minutes

The thrust-to-weight ratio (TWR)is a measurement in aviation that indicates the relation between the thrust produced by the motors and the weight of the vehicle. It indicates how effectively the drone can lift off, hover and accelerate

TWR=

- If TWR < 1, the drone cannot take off.
- If TWR > 1, the drone can take off and accelerate.

Each 2200 KV motor paired with a 10×4.5 propeller produces approximately **1000 gf** (grams-force) of thrust. For four motors:

Total thrust=1000×4=4000 gf=4 kgf

The drone’s total weight is **1400 g = 1.4 kg**.

Since thrust and weight in grams-force or kilograms-force relate directly, you can calculate TWR simply by dividing mass in kilograms (or grams) because 1 kgf=9.81 N1, so the gravitational constant cancels out. Thus, TWR= =2.85 This means the drone can safely and easily take off and accelerate, as the thrust produced is almost three times its weight.

3.4 System Wiring Diagram

The following diagram presents the complete wiring setup of the UAV system. It shows how the main components are connected in terms of both power and data communication. The Raspberry Pi 4B receives regulated 5V power through a step-down converter linked to the power distribution board. It connects to the Pixhawk flight controller via USB to handle MAVLink communication.

Environmental sensors such as the SHT3x and MQ-135 are linked to the Raspberry Pi through its GPIO pins using I2C and analog inputs. A separate serial interface handles data from the 360° LiDAR, while the onboard USB camera streams live video directly to the Pi. Communication with the ground station is achieved through either Wi-Fi or a LoRa module, depending on mission requirements.

This configuration ensures stable power supply and organized data routing, helping to maintain reliable performance throughout the flight.

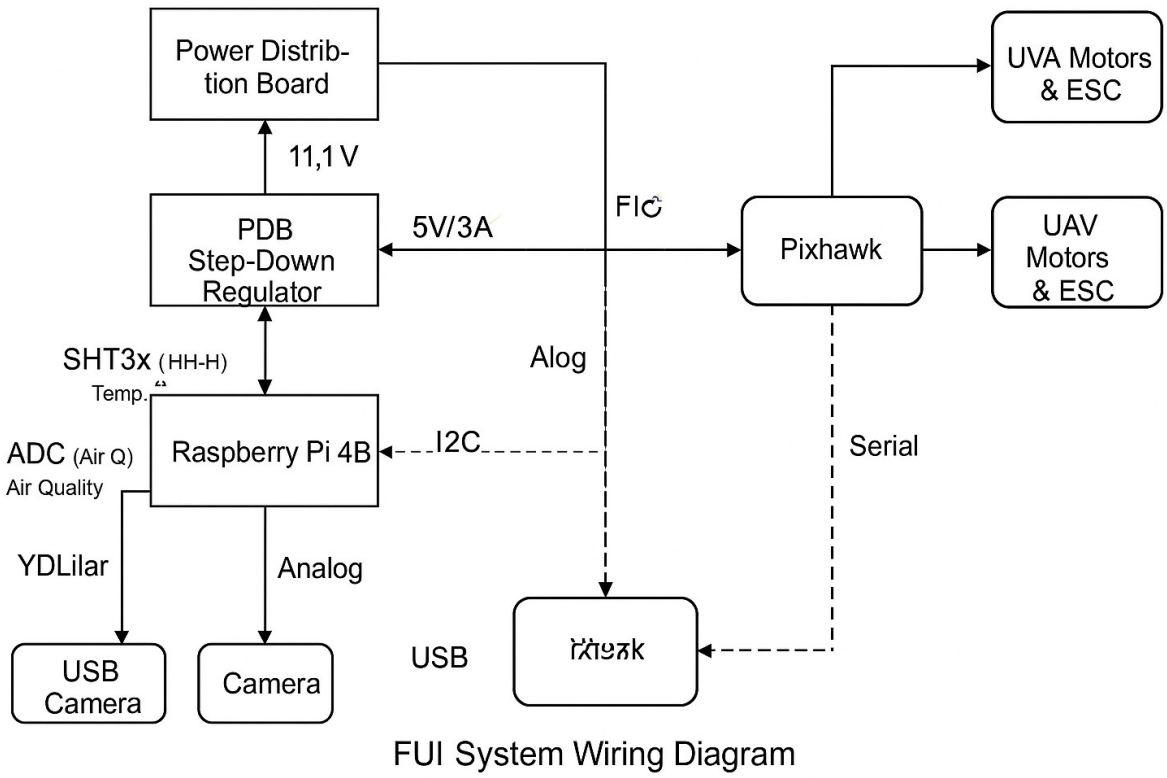

Figure 37 – Full schematic circuit

3.5 Conclusion

This chapter provided a comprehensive UAV system overview by dividing the system into two subsystems—the drone and the ground station—followed by a detailed discussion of the drone's hardware architecture, which supports autonomous flight, sensing, and communication. Through the careful selection and integration of the flight controller, companion computer, sensors, actuators, and power system, it achieves a robust platform capable of executing complex missions such as disaster reconnaissance and search-and-rescue operations.

The chapter concludes with detailed calculations of system power consumption at different battery charge levels, optimizing power performance to maximize flight time, as well as thrust-to-weight ratio calculations to ensure safe and reliable takeoff.

Chapter 4: Software Implementation

This chapter provides a detailed overview of the software architecture for the UAV system, which ensures the coordination and functionality of the various components critical for mission success. Software implementation is essential for enabling the UAV's subsystems to operate reliably, efficiently, and autonomously.

The software system is composed of three primary components:

1. **ArduPilot on the Pixhawk2.4.8:** ArduPilot is responsible for controlling the UAV's core flight functions. It manages sensor data processing, flight parameter tuning, and various flight modes. This component enables diverse flight configurations while maintaining high reliability during operations.
2. **ROS on Raspberry Pi 4:** Robot Operating System (ROS) facilitates efficient data handling and enhances the modularity of the UAV system. It interfaces with the UAV's sensors and actuators, enabling real-time data collection and processing. ROS is also responsible for handling obstacle avoidance calculations and their execution.
3. **Flask Server on the Ground Station:** The ground station receives real-time UAV data streams and processes these inputs using machine learning algorithms for fire detection and victim localization. It sends mission files back to the UAV and provides real-time visualization of mission progress, sensor data, and system status, all while ensuring the UAV's autonomy and operational efficiency.

These components are seamlessly integrated via the **MAVROS** interface, which connects the Raspberry Pi to the Pixhawk, and the **ROSbridge** protocol, which links the Raspberry Pi to the Flask server. This architecture ensures reliable data flow between the UAV and the ground station, creating a cohesive and smoothly operating system.

The following section will provide specific implementation details for each software layer necessary for the UAV to execute its mission effectively.

4.1 Software flowchart

This diagram illustrates the process flow of a fully autonomous UAV mission, involving components such as the Ground Station, Companion Computer, and Flight Controller. Key actions include mode detection, obstacle data handling, mission launching, data transmission, machine learning applications, and mission completion detection. The diagram captures the interactions between various subsystems and the sequence of operations required for a UAV to autonomously navigate and complete its mission.

Autonomous UAV Mission Execution SOP

1. Initialize the system components:

    1.1. Start ROS Initialization.

    1.2. Initialize MAVROS.

    1.3. Initialize Flask.

    1.4. Initialize ROSBridge.

2. Prepare for mission launch:

    2.1. Select Coordinates.

    2.2. Input Parameters.

3. Launch the mission:

    3.1. User initiates Mission Launch.

    3.2. Generate the mission plan.

    3.3. Transmit mission data.

    3.4. Push mission to the UAV.

4. Begin flight operations:

    4.1. Detect Auto Mode.

4.2. Launch Sequence initiated.

4.3. Monitor mission progress.

5. Handle obstacle detection and avoidance:

5.1. Gather obstacle data.

5.2. Execute Obstacle Detection.

5.3. If obstacles detected, apply Avoidance Logic.

5.4. Execute necessary commands for avoidance.

6. Navigate waypoints:

6.1. Follow Waypoint Navigation.

6.2. Upon reaching the last waypoint, initiate Landing.

7. Complete the mission:

7.1. Detect Mission Completion.

7.2. Display results on the UI.

8. Finalize data operations:

8.1. Collect and transmit data.

8.2. Apply Machine Learning Applications.

8.3. Confirm data receival and processing.

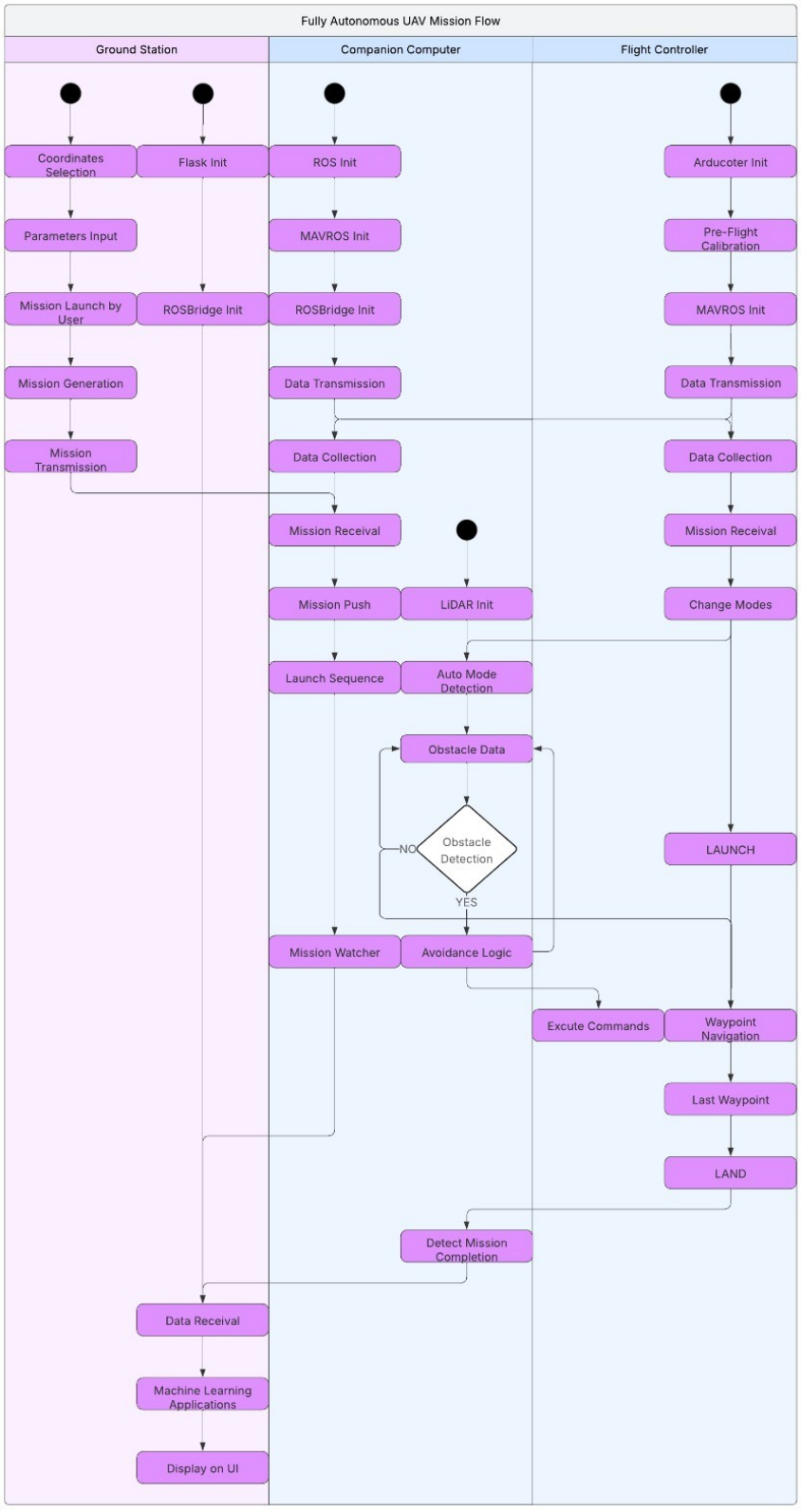

Figure 38 – Full schematic flowchart

4.1 ArduPilot

ArduPilot is a robust, open-source firmware platform widely used for controlling various unmanned vehicles, including drones, ground vehicles, and boats. In this project, ArduPilot manages the quadcopter's core flight operations. The firmware supports mission execution, waypoint navigation, and integrates multiple sensors and actuators essential for UAV mission success. Its flexible architecture allows for extensive customization, ensuring compatibility with various sensor configurations and flight modes.

4.1.1 ArduCopter Key Features

ArduCopter offers several flight modes, each designed to provide varying levels of control, ranging from manual piloting to fully autonomous operations. These modes enable the UAV to adapt to specific mission requirements and environmental conditions:

Stabilize Mode: In this mode, the UAV is manually controlled via the RC transmitter. The operator manages pitch, roll, and yaw inputs, while the system automatically maintains level flight.

Auto Mode: In Auto mode, the UAV follows a pre-defined mission, typically specified by a waypoint file in JSON format. This is a fully autonomous mode where the UAV navigates to each waypoint without operator intervention.

Guided Mode: Guided mode allows the UAV to be controlled through commands from the ground station or a companion computer. This mode provides precise path control, which is particularly useful for obstacle avoidance or dynamic path adjustments.

Return-to-Launch (RTL) Mode: This safety mode commands the UAV to automatically return to its home position in case of lost communication, low battery, or other critical emergency conditions.

These modes can be switched during flight either by the operator using the RC transmitter or via MAVROS commands sent from the Raspberry Pi companion computer.

4.1.2 Sensor Integration

ArduCopter's integration with the Pixhawk 2.4.8 flight controller enables the UAV to collect and process critical data from multiple sensors, ensuring reliable flight control and navigation. This section describes how the Pixhawk utilizes data from the UAV's key sensors to maintain safe and efficient flight:

Inertial Measurement Unit (IMU): The IMU detects the UAV's orientation and accelerations. It provides essential data for stabilizing the UAV by adjusting pitch, roll, and yaw in response to motion dynamics.

GPS and Compass: These sensors provide geolocation and heading data, which are vital for waypoint navigation and ensuring the UAV maintains its correct course during autonomous flight. The GPS must lock onto at least seven satellites, with a Horizontal Dilution of Precision (HDOP) below 2.0, to achieve accurate navigation.

Barometer: The barometer measures atmospheric pressure and is used to regulate the UAV's altitude. This is crucial for maintaining stable and consistent flight levels.

Landing LiDAR: The integrated LiDAR sensor measures real-time distance to the ground, enabling precise and safe landings during the UAV's descent.

By leveraging data from these sensors, ArduCopter effectively manages key flight parameters, including altitude, position, and stability, ensuring smooth and controlled UAV operations.

4.1.3 Pixhawk 2.4.8

The Pixhawk 2.4.8 flight controller serves as the core of the UAV system, running the ArduPilot firmware to manage all flight functions. This section details how ArduPilot is installed and configured on the Pixhawk, along with the calibration of sensors, Electronic Speed Controllers (ESCs), and motors. ArduPilot is deployed on the Pixhawk using the **Mission Planner** interface, which provides a graphical environment for firmware installation and configuration. Mission Planner allows users to install the ArduPilot firmware and set critical flight parameters required for safe and autonomous operations. Accurate sensor calibration is essential for reliable flight performance. The following calibrations must be performed during setup:

- **Compass Calibration:** Ensures accurate heading data by calibrating both the internal Pixhawk compass and the external GPS compass. The calibration process involves rotating the UAV in multiple orientations until the system records acceptable offset values.
- **Accelerometer Calibration:** Guarantees accurate measurements of acceleration and orientation. The UAV must be positioned in various orientations (level, nose up, nose down, left side, right side) to complete this calibration.
- **ESC Calibration:** ESCs regulate motor speeds. Calibration ensures synchronized motor responses by setting the throttle stick to maximum, powering up the UAV, and allowing the ESCs to register the maximum throttle range.

4.1.4 Safety and Failsafe Protocols

Safety protocols are critical for minimizing operational risks. **ArduCopter** incorporates multiple failsafe mechanisms to protect the UAV in the event of system errors or unexpected conditions:

Battery Failsafe: If the battery voltage drops below a predefined threshold (e.g., 10V), the UAV automatically initiates a **Return-to-Launch (RTL)** procedure or lands immediately to prevent a crash.

Communication Failsafe: If the connection between the Pixhawk and the ground station is lost, the UAV either returns to the last known safe location or executes an emergency landing.

These failsafe mechanisms significantly reduce the risk of accidents and ensure the UAV remains controllable under adverse conditions.

4.1.5 Parameter Configuration and Customization

The ArduPilot firmware provides an extensive parameter list that enables operators to fine-tune the UAV's flight behavior, sensor responsiveness, and safety features. Correct parameter configuration is vital for achieving optimal UAV performance tailored to specific mission requirements. Key parameters include:

EKF (Extended Kalman Filter) Tuning: The EKF algorithm fuses data from the IMU, GPS, and magnetometer to provide accurate estimations of the UAV's position, velocity, and orientation.

Failsafe Parameters: These settings define acceptable GPS drift limits, battery voltage thresholds, and conditions for automatic mode switching or arming denial to prevent unsafe operations.

Motor Output Parameters: These parameters control how control inputs are translated into motor outputs. Settings such as **Throttle Acceleration Limits**, **Yaw Authority**, and **Motor PWM limits** ensure precise motor control and response.

Proper tuning of these parameters ensures that the UAV operates safely, efficiently, and is adaptable to a wide range of mission environments and conditions.

4.2 Robot Operating System

The Robot Operating System (ROS) is an open-source middleware framework developed by Willow Garage in 2007. ROS simplifies and organizes the software architecture for robotic systems. It provides a collection of tools and libraries designed to help developers build modular, functional, and flexible software components that can communicate and collaborate effectively within a multi-component system.

4.2.1 ROS Architecture

ROS provides an abstraction layer between software and hardware, enabling the management of advanced robotic behavior through several core services. At its core, ROS is composed of multiple independent processes known as nodes. This modular design allows each node to perform a specific function, such as sensor data acquisition, state estimation, control algorithms, communication handling, or actuator control. Nodes can be developed in C++ or Python, supported by a vast set of libraries.

ROS nodes communicate through a publish/subscribe messaging system using topics as communication channels. A node can publish messages to a topic, and any other nodes subscribed to that topic will receive the data. In addition to topics, ROS offers services and actions as communication mechanisms, allowing nodes to exchange information and coordinate tasks in a structured and synchronized manner. Furthermore, ROS features a centralized parameter server for dynamic system parameter management. This server allows developers to configure and adjust node behavior at runtime without altering the source code, providing flexibility for tuning operational characteristics.

ROS is also equipped with powerful tools for diagnostics, logging, visualization (e.g., RViz), and data recording (Rosberg). These tools facilitate debugging, testing, and post-mission analysis, ensuring reliable system development and maintenance. The structure of ROS enables comprehensive management of planning, perception, and control processes within the UAV system.

To simplify the management of complex robotic systems, ROS uses launch files, which are configuration scripts (typically in XML or Python format) that automate the execution of multiple nodes, load parameters, and manage logging. Launch files streamline system initialization by enabling the simultaneous start of various components, ensuring they work together cohesively.

Finally, ROS organizes its software into packages, which bundle together nodes, libraries, configuration files, and other necessary resources. A package can contain everything required to implement a specific functionality, including all associated nodes, launch files, and parameter configurations, promoting modular and reusable software development.

4.2.2 ROS 2 Workspace

This project uses ROS 2 Humble, running on the Ubuntu 22.04.5 LTS operating system. Built upon the foundation of ROS 1, ROS 2 introduces substantial improvements in performance, capabilities, and features. Its modular design, based on a distributed architecture, allows it to operate seamlessly in dynamic environments. By leveraging the Data Distribution Service (DDS) protocol, ROS 2 enables nodes to exchange real-time data directly with one another, avoiding the bottlenecks that can arise from relying on a centralized management system.

A ROS 2 workspace defines the structure for managing and organizing multiple packages during the development of ROS 2 applications. It includes key components such as source code, build outputs, installation files, and log data. These elements are important for the successful development, testing, and deployment of ROS 2 applications.

The Table 4 below outlines the key components of a ROS 2 workspace, providing descriptions of their respective functions and roles in the development process

Table 4 - ROS 2 Workspace components

Component	Description
Nodes	Code written in Python or C++ that performs specific tasks within the package.
Launch files	Scripts (usually in Python) that define how to start nodes and configure their parameters.
package.xml	Metadata file defining the package's name, version, description, maintainer, and dependencies for build, run, and test.
setup.py	Python-specific file that organizes the code structure, mapping source code to executable nodes and console scripts.
Test files	Files used to verify the dependencies in package.xml and ensure proper node functionality during testing.
Build directory	Generated by Colcon build; contains build artifacts like object files and dependency information and tracks the build process.
Install directory	Holds installed libraries, executable tools, and setup files necessary for ROS 2 packages to run properly.
Log directory	Stores logs from the build and test processes, including errors and warnings, which are crucial for debugging and troubleshooting issues.

4.2.3 ROS Algorithm

The core logic of the companion computer in this UAV system is implemented using several ROS packages and nodes. These packages provide the necessary functionality for handling sensors, communicating with the Pixhawk, and executing fully autonomous missions by ensuring obstacle avoidance. Each package runs directly on the Pi at startup to process real-time data, control the UAV's behavior, and ensure safe and efficient operations. Below is a detailed description of each package and its specific role within the system.

4.2.3.1 Drone Package

The Drone Package is responsible for managing key components of the UAV's operations, including camera control, environmental sensor data, and mission reception. This package integrates the environmental sensor and USB camera streaming nodes, ensuring that the UAV gathers necessary data and transmits it to the station. It handles the following functionalities:

Camera Control: Manages the USB camera stream, capturing live video and publishing it to a designated ROS topic for transmission.

Environmental Sensor Data Manipulation: Responsible for reading and publishing environmental sensor data, such as temperature, humidity, and air quality. The SHT3X sensor (for temperature and humidity) and MQ-135 sensor (for air quality) are interfaced with the Raspberry Pi and publish this data to appropriate ROS topics for real-time monitoring on the ground station.

Mission File Reception: Listens for mission files (in JSON format) that contain waypoints and mission parameters. These files are published to a ROS topic, where they are picked up by the Navigator Package for mission execution.

ROS-Bridge Transmission: Facilitates communication with the web interface through ROS-bridge, ensuring that data such as sensor readings and the camera feed are continuously transmitted to the ground control station.

4.2.3.2 Navigator Package

The Navigator Package is tasked with executing the autonomous flight mission, starting from the UAV's current position to a sequence of waypoints defined in the mission file. This package manages communication with the Pixhawk through MAVROS and ensures the UAV follows its mission autonomously.

Mission File Upload to Pixhawk: Uploads the mission file received from the Drone Package to the Pixhawk. The mission file contains an ordered list of waypoints that define the flight path. These waypoints include crucial parameters like latitude, longitude, altitude, and yaw, which are sent to the Pixhawk for execution.

Mission Sequence Execution: Once the mission file is successfully uploaded to the Pixhawk, the Navigator Package ensures that the UAV starts following the waypoints in the mission file. It switches the UAV to the appropriate flight mode (e.g., Auto, Guided) and commands the Pixhawk to execute the mission autonomously.

Communication with MAVROS: Communicates with MAVROS to control the flight mode and manage telemetry data. It listens to GPS data, flight status, and mission updates.

4.2.3.3 Obstacle Avoidance Package

The Obstacle Avoidance Package ensures that the UAV can navigate safely by avoiding obstacles in its path. It processes LiDAR data to generate real-time 360-degree scans of the environment and adjusts the UAV's flight path accordingly.

LiDAR Data: The LiDAR sensor continuously scans the environment and provides distance measurements. The Obstacle Avoidance package receives this data and uses it to create a map of the surrounding obstacles.

Path Planning: Based on the LiDAR data, the Obstacle Avoidance package calculates the optimal flight path to avoid detected obstacles. If an obstacle is detected in the UAV's current path, the package adjusts the trajectory to find a safer route.

Flight Mode Switching: When an obstacle is detected, the node changes the flight mode from Auto to Guided, allowing the Raspberry Pi to take control and send the necessary commands for obstacle avoidance, ensuring the UAV continues its mission safely.

4.2.3.4 MAVROS Package (Native ROS Package)

MAVROS is a native ROS package that provides the communication bridge between ROS and the Pixhawk flight controller, enabling the UAV to communicate with ROS nodes for both flight control and telemetry collection.

Flight Mode Control: MAVROS sends MAVLink commands to the Pixhawk, allowing the ROS system to switch the UAV's flight mode.

Telemetry Data Collection: MAVROS subscribes to the relevant MAVLink messages from the Pixhawk, collecting telemetry data such as GPS coordinates, altitude, battery status, and sensor readings.

Mission File Upload and Execution: MAVROS facilitates the uploading of mission files to the Pixhawk, containing waypoints and mission instructions. MAVROS ensures that the UAV's flight modes and mission parameters are properly executed.

Safety and Emergency Protocols: MAVROS enables emergency control features, such as return-to-launch (RTL) and emergency landing. If the UAV experiences a critical failure, MAVROS ensures that the UAV safely returns to its launch point or lands autonomously.

4.3 Ground Station Server

The Drone Operations Center is designed to provide an intuitive and interactive platform for controlling and monitoring the UAV in real time. It consists of two main components: the Backend Server and the Frontend Interface. The backend handles the UAV control logic, data processing, critical safety controls, and communication with the hardware, while the frontend visualizes these functionalities in a user-friendly platform, offering intuitive control and real-time monitoring for the operator.

4.3.1 Flask Backend Server

The backend of the Drone Operations Center is powered by a Flask server that handles communication between the web interface and the UAV. It uses Socket.IO to facilitate the transfer of mission data, telemetry, and video feeds. The backend also integrates with ROS 2 to manage the UAV's flight control and sensor data.

Core Functions and Workflow:

Initialization: The backend initializes the necessary configuration parameters, such as the ORS API key for route planning, the Raspberry Pi IP address for communication, and the mission storage paths. It configures secure remote access to the UAV's Raspberry Pi using Paramiko (SSH connection), ensuring that the mission files are correctly transferred and executed.

Route Planning: The backend integrates the OpenRouteService (ORS) API to calculate and generate routes for the UAV, considering terrain and elevation data. A function provides the route's elevation profile, which is critical for mission planning.The scan pattern generation function creates circular scan patterns for missions requiring area coverage, such as search and rescue or environmental monitoring.The backend generates mission JSON files that define waypoints and operational parameters like scan radius, cruise altitude, and scan altitude. These missions are built as JSON files and transferred to the Raspberry Pi.

Real-Time Data Communication: The backend leverages ROSbridge to facilitate seamless communication between the Raspberry Pi (running ROS 2) and the Flask web interface. ROSbridge provides a WebSocket interface for ROS, allowing for real-time exchange of data between the drone unit and the frontend.

By subscribing to relevant ROS 2 topics, the backend continuously relays mission status updates, telemetry data (such as GPS position, altitude, and sensor readings), and error messages to the frontend using Socket.IO. Thanks to Socket.IO, real-time updates are pushed from the backend to the frontend with minimal delay. This includes continuous updates for GPS status, battery levels, sensor data, and flight progress.

The backend handles mission file transfers to the Raspberry Pi using SCP (Secure Copy Protocol), ensuring that mission files are securely transferred and verified on the Raspberry Pi for mission execution. After transferring the mission, the backend remotely triggers the relevant ROS service on the Raspberry Pi to start the mission, return to launch, or land the drone.

Machine Learning: In the video streaming and object detection pipeline, the backend utilizes YOLO (You Only Look Once) models for real-time object detection, including human and fire detection. These tasks are computationally intensive, especially with real-time video streams, so task offloading strategies are employed to ensure the Raspberry Pi isn’t overloaded, optimizing both efficiency and performance.

Human Detection: The backend uses a pre-trained YOLO model specifically filtered for human detection. The model processes each frame from the UAV's camera feed to identify human figures. The detection process involves analyzing the image feed, running each frame through the YOLO model, and annotating the detected humans with bounding boxes.

Fire Detection: A custom-trained YOLO model is used to detect fire in the video feed. This model was trained on a specialized fire detection dataset, ensuring it can reliably identify fire under various environmental conditions. By training the model on a diverse set of fire images, it can distinguish fire from other visual disturbances in the environment, such as smoke or lighting artifacts. The UAV’s Raspberry Pi only streams the video feed to the companion computer, where the human and fire detection models are executed. The intensive task of running these YOLO-based detection models is offloaded from the Raspberry Pi to more powerful hardware, ensuring the Pi remains responsive for flight control and other critical operations.

4.3.2 Frontend Interface

The frontend is built using HTML and JavaScript to create an interactive web interface that allows the user to control the UAV, monitor mission progress, and view real-time data. It communicates with the Flask backend through Socket.IO to receive mission updates, telemetry, and video feeds.

Providing a real-time map using the Leaflet library to track the UAV's location, set mission coordinates, and display real-time GPS data.

Allowing the operator to view live video feeds from the UAV's camera. The video feed can be switched between different modes, such as live feed, human detection, or fire/smoke detection, using the YOLO models integrated into the backend.

Displaying real-time telemetry data, including GPS coordinates, temperature, humidity, air quality (AQI), and wind speed. This information is updated dynamically as it is received from the UAV via Socket.IO. The GPS status is continuously monitored and displayed, indicating the number of satellites, HDOP (Horizontal Dilution of Precision), and fix status.

The mission control panel allows the user to specify mission parameters such as scan radius, cruise altitude, and scan altitude. The user can then launch the mission by clicking the *Launch Mission* button, which sends the mission parameters to the backend. The system also supports custom mission uploads. Users can upload a JSON mission file, which is validated and transferred to the UAV for execution. It also features dedicated *Return-to-Home (RTL)* and *Emergency Landing* buttons.

4.4 Conclusion

Chapter 4 described the software implementation of the UAV system, focusing on the integration of the three core components: ArduPilot, ROS, and the Flask server. Each component plays an essential role in the system's functionality. The system ensures autonomous flight, reliable mission execution in dynamic environments, and continuous data collection. The ground station server makes interacting with the UAV intuitive, offering real-time updates on position and environmental data throughout mission execution. This software architecture enables the UAV to execute complex tasks while providing operators with crucial real-time situational awareness.

Chapter 5: ASSEMBLY, TESTING, and VALIDATION

The outcome of this engineering work is achieved through the precise assembly, integration, and validation of the autonomous UAV system. Unlike theoretical models or lab-only prototypes, this platform was designed to function reliably in the challenging and diverse environments found in Lebanon, from the rugged pine forests of Akkar, where wildfires are common in summer, to the dense urban areas of Beirut, where post-disaster assessments are critical.

This chapter details the full assembly process, explains the integration strategy applied to combine all subsystems, and describes the structured multi-stage testing carried out to verify the system's performance. The testing phases were designed not only to check each component but also to evaluate the UAV as a complete system, ensuring that it can perform efficiently as an autonomous tool for reconnaissance in demanding, real-world conditions.

5.1 Mechanical and Electrical Assembly

5.1.1 Structural Integration and Layout

The UAV system is built on an F450 quadcopter frame, inspired by the DJI design, which is known for its stable X-shaped geometry, modular ABS arms, and strong mechanical resistance to torsion. This choice helped in efficiently arranging the onboard components while maintaining a proper center of gravity ─essential for stable hovering and responsive yaw control. Table 5 sums up the integration layout and purpose of each key module on the F450 frame.

Photo

Table 5 - Components location

Components	Mounting Location	Purpose
Pixhawk 2.4.8	Center of the frame, mounted on foam pads	Dampens vibrations to protect IMU data quality and ensure stable flight control
Raspberry Pi 4 B	Rear landing gear area (3D-printed platform)	Handles onboard computing; foam mounting reduces vibration
ESCs	Along each arm	Exposed to airflow for cooling; easy to maintain and replace
GPS Modules	Elevated on raised supports, away from motors	Minimizes EMI and improves satellite signal quality
LiDAR Sensor	Mounted on top of the UAV	Provides obstacle data in clear line-of-sight; avoids false detection
Camera	Front-facing on chassis	Captures stable video for streaming, human detection, and fire detection.
Landing LiDAR (TFT-Mini)	Mounted at back of chassis, facing downward	Measures altitude for precise landing control
Power Module	Rear section of the drone	Supplies stable power and monitors battery status
Regulator	Back of chassis	Regulates voltage to onboard electronics
Air Quality Sensor (MQ-135)	Back center frame	Clean airflow for accurate

		readings (avoids prop wash)
Temp/Humidity Sensor (SHT3-X)	Underside of drone	Isolated from heat sources for reliable measurements
Safety Switch	Side frame (accessible position)	Emergency power cutoff for motors during handling/maintenance
Wiring & Cables	Secured with spiral wraps and heat-shrink tubing	Prevents wear, protects connections, and ensures long-term reliability

5.1.2 Communication and Telemetry Infrastructure

The UAV's communication system was designed to establish reliable data exchange between the onboard processors, sensors, and the ground control station. The architecture ensures stable and low-latency communication throughout the mission phases.

- **MAVLink Protocol**

  Used to structure telemetry messages exchanged between the Pixhawk and the Raspberry Pi. These **MAVLink** telemetry messages are transmitted via a dedicated UART connection configured at a baud rate of 57,600 to optimize communication stability and transmission rate. The **MAVROS** package running on the Raspberry Pi bridges these **MAVLink** messages into ROS topics, enabling seamless integration between flight control and onboard sensor processing.

- **Raspberry Pi's USB 2.0/3.0 ports**

  Assigned to the **YD LiDAR X4 PRO** and the **720p webcam**, ensuring sufficient data bandwidth to handle high-frequency LiDAR point clouds and real-time video streams simultaneously, without risk of data congestion or performance degradation.

- **The MQ-135 gas sensor**

Allocated to the **MCP3008 ADC**, the analog output of the **MQ-135** gas sensor was converted via the Raspberry Pi's SPI interface. This configuration allowed the system to publish real-time air quality data as ROS topics for onboard sensor monitoring.

- **Wifi Protocol**

Configured in client mode, the **Wi-Fi interface** enabled communication between the UAV and the ground station, ensuring reliable transmission of telemetry and sensor data throughout the mission.

5.1.3 Thermal Management

- **Heat generation sources:** ESCs, DC-DC regulator, Motors, and the Raspberry Pi generates substantial heat during flight, particularly under high power loads. Overheating can cause performance issues or permanent damage.
- **Passive dissipation via frame:** Nylon-reinforced UAV frame naturally aids in dissipating heat across the chassis, helping reduce thermal buildup without adding active cooling components.
- **Component spacing for airflow:** ESCs and motors are strategically spaced to promote airflow during flight, enhancing cooling through natural convection.
- **Aluminum heat sink on DC-DC regulator:** Aluminum heat sink is mounted on the DC-DC regulator to improve heat dissipation and maintain stable 5V output by reducing thermal stress during extended operations.
- **Thermal silicon conductive pads on Raspberry Pi ICs:** Enhance heat transfer from the processor and other integrated circuits to surrounding surfaces, stabilizing temperatures when running multiple ROS nodes simultaneously.
- **Future enhancements – digital temperature monitoring:**

Integrating temperature sensors could enable real-time thermal feedback, allowing dynamic management strategies such as automated throttling, shutdowns, or alarms when high temperature thresholds are reached, thereby improving safety and system longevity.

5.1.4 Electrical Safety and Redundancy

Ensuring electrical safety is essential for maintaining flight stability and protecting sensitive onboard components. The power module plays a key role by precisely regulating the voltage and monitoring current levels to prevent overvoltage or overcurrent conditions that could damage the system.

The Pixhawk flight controller is equipped with built-in failsafe mechanisms that can trigger an automatic landing or power shutdown in the event of signal loss or system faults. However, the current configuration does not include hardware redundancy, such as backup power supplies or dual flight controllers. Incorporating such redundancy in future designs could enhance the system's reliability, especially for missions where operational continuity is important.

5.1.5 Weight Distribution

The complete UAV, fully assembled with all components, wiring, and protective mounts, weighs approximately 1400 g. This measured weight is important to confirm that the drone can achieve the expected lift and maintain a safe thrust-to-weight ratio, ensuring performance as calculated during the design phase. Equally important is the distribution of this weight across the frame. The battery pack is mounted low to lower the center of gravity, enhancing overall stability. Meanwhile, heavier components like the Pixhawk and LIDAR are installed close to the frame’s center, helping to keep the center of mass aligned with the geometric center. This balanced layout, supported by the X-shaped frame design, reduces unwanted tilting or oscillations in flight and ensures the drone responds predictably to control inputs, even during sharp maneuvers or in turbulent conditions.

5.1.6 Environmental Robustness

The UAV is designed to operate reliably in diverse and often harsh environments, as typically encountered in post-fire reconnaissance missions. The selected components are capable of withstanding wide temperature ranges and moderate levels of humidity.

To further enhance durability, the system will be enclosed in a dustproof and waterproof casing, ensuring protection against environmental hazards. This design consideration aims to maintain consistent performance and safeguard the hardware even in adverse weather conditions.

5.1.7 Limitations and Future Hardware Improvements

Based on performance evaluations, several limitations and opportunities for hardware enhancement have been outlined:

- **Limited flight time**

  Due to current battery capacity, restricting missions to under 20 minutes at moderate throttle.

- **Sensor resolution and range**

  The LIDAR and air quality modules could be improved with higher-end alternatives to enhance environmental mapping accuracy.

- **Lack of hardware redundancy**

  Such as dual flight controllers or backup power systems, reduces reliability; integrating these would improve mission safety but add weight and complexity.

- **Discrete electronic components**,

  Including voltage regulators, sensor breakout boards, and power management circuits, could be consolidated into a **custom PCB** to:

  1. Reduce wiring clutter and assembly time
  2. Improve signal stability and reduce electromagnetic interference
  3. Increase durability and system maintainability

- D**edicated companion PCB**

Centralize sensor connections and peripheral interfaces, streamlining integration into the UAV's frame.

- **Future iterations may consider:**

1. **Lightweight, high-capacity batteries** for extended flight time
2. **Upgraded sensor suites** with higher accuracy and longer range
3. **More powerful companion computers** like the NVIDIA Jetson series to support advanced onboard AI and autonomy.

5.2 Flight check steps

Before each flight, it is important to verify that all essential software components and hardware interfaces are fully operational to ensure a safe and successful mission. The companion computer automatically launches the necessary ROS environment and related nodes at boot. The following steps outline the systematic procedure to confirm that all nodes, communications, and sensors are functioning correctly before proceeding with mission execution.

1. **Verify MAVROS and Pixhawk Communication on Boot**
    - Confirm that the mavros node starts automatically when the Raspberry Pi boots up.
    - Check that the communication link between mavros and the Pixhawk is successfully established.
    - Monitor the state information published by mavros to verify the connection status is active.
    - Ensure position and telemetry data from the Pixhawk are being received through mavros.
    - Visually confirm the Pixhawk's onboard LEDs indicate proper power and communication status.

2. **Verify Rosbridge Connection Between Ground Station and Raspberry Pi**

- Ensure the ROSBridge server is running automatically on the Raspberry Pi after boot.

- Confirm that the ground station server establishes a successful network connection with the ROSBridge server on the Pi.

- Monitor communication by checking the exchange of heartbeat or status messages between the ground station and the Pi.

- Verify that commands and data can be sent and received smoothly through this connection, ensuring real-time interaction.

3. Verify GPS 3D Fix from Pixhawk

- Check the Pixhawk's onboard LED to determine GPS status; **solid blue** light typically indicates that a valid 3D fix has been acquired.
- Cross-verify the GPS fix through MAVROS telemetry to ensure the position data is being received and is consistent.
- Confirm the GPS signal is stable and locked before initiating waypoint navigation.
- Always wait for both visual and software indicators to confirm a solid 3D fix, ensuring reliable and accurate flight positioning.

To help interpret the Pixhawk's visual signals during pre-flight inspections, the table below outlines the significance of each LED color and pattern

Table 6 - Pixhawk LED Status Indicators

LED Color	Flight State	Description
Flashing Red	Disarmed, no GPS fix	Vehicle is powered but GPS lock has not been acquired
Flashing Blue	Disarmed, acquiring GPS	GPS is searching for satellites
Solid Blue	Disarmed, GPS 3D fix	A valid 3D GPS fix has

	acquired	been achieved
Flashing Green	Disarmed, ready to arm	System has passed pre-arm checks and is ready
Solid Green	Armed and ready	Vehicle is armed and can begin flight
Breathing Green	In flight, system healthy	Vehicle is armed and flying; no critical errors
Flashing Yellow	Warning or failsafe active	Indicates a pre-arm error, failsafe, or RC signal lost
Solid Red	Critical error	Serious hardware, configuration, or safety issue; do not proceed with flight

4. Verify Navigator Package and Mission Execution

- After boot, the **navigator_pkg** remains idle, waiting for mission instructions in the form of a JSON file.
- The ground station sends this JSON file to the Raspberry Pi over the Rosbridge connection. It contains the full set of waypoints and mission commands.
- Once received, the package parses the file and activates its internal logic to begin autonomous navigation.
- The navigator node then sets the appropriate flight mode through MAVROS and begins sending waypoint commands to the Pixhawk.
- Waypoint traversal is monitored in real-time, allowing the UAV to follow the defined mission path accurately.
- Integrated within the package is an obstacle avoidance logic layer, which listens to LIDAR inputs and continuously checks for obstructions along the flight path.
- If a predefined safety threshold is crossed (e.g., an object detected within a minimum range), the avoidance logic overrides the waypoint flow temporarily to initiate evasive action.

- Once the path is clear, the navigator resumes normal waypoint tracking and continues the mission.

5.3 Testing and validation

Following the selection and assembly of all hardware components, a series of integration tests were performed to validate the system configuration and verify proper communication between all subsystems. Initial bench tests focused on confirming sensor outputs, ensuring stable communication between the Pixhawk and Raspberry Pi, and checking that all modules received appropriate power.

Subsequent flight tests were conducted in controlled environments to evaluate system stability, sensor precision, and real-time data transmission to the ground station. Issues such as signal interference and occasional power fluctuations were identified and resolved by optimizing the wiring layout and incorporating filtering components where needed.

5.3.1 Manual flight tests

After calibrating the ECSs, compass, and IMU in the Pixhawk series, manual flight tests were conducted using the RC transmitter to ensure that all the ESCs are delivering the same PWM to the motors, so when throttle input is varied, all the motors will perform at the same speed, providing a stable lift. In addition, a test was conducted by catching the landing gear, lifting the UAV overhead, giving a throttle input, and trying to move it in all directions to ensure that the UAV would adjust itself in order to stay stable. These tests will ensure good IMU calibrations. While conducting these tests, the UAV faced multiple crashes, which enabled learning more and more about how to control the UAV. After all the tests and failures in manual flight, the UAV was finally stable and had self-adjustment, making it ready for autonomous flight, although the UAV had a slight normal backward drift.

5.3.2 Environmental sensor tests

Several tests were made for the environmental sensor and the camera video stream by powering the drone and checking if the data were transmitted to the ground station server on boot

using Rosbridge. The GPS data coming from the Pixhawk, which appears on the ground station map as a UAV figure, is almost accurate within a 2-meter range. As for the environmental sensor, the tests ensure the response of the air quality, temperature, and humidity data to environmental changes. The readings were validated through a calibrated reference device that measures humidity and temperature.

The onboard camera delivers real-time video streaming with no noticeable latency, allowing uninterrupted visual monitoring during flight. To optimize onboard processing load, fire and human detection tasks are offloaded to the ground station server. Human detection is performed using the YOLOv8 algorithm, which processes incoming video frames to identify and localize individuals within the UAV's field of view. Fire detection uses a pretrained convolutional neural network model specifically trained to detect fire events in real-time video input. This distributed architecture ensures fast video transmission from the drone and efficient, high-accuracy inference on the server side.

The table below shows different sensor data readings alongside their corresponding actual readings.

PHOTO FOR GPS FROM SERVER

Table 7 - Sensor measurement

Sensor	Sensor reading	Actual reading	Accuracy
GPS (satellites)	6-9	10	±3 meters
Humidity (%)	70-75	73	0.5%
Temperature (℃)	26-29	30	±2–3%
Air quality (aqi)	99-102	105	4.5 AQI

These testing phases confirmed the reliability of the hardware architecture and revealed opportunities for further optimization, such as refining sensor placements and adjusting calibration parameters to improve overall flight performance.

5.3.3 Autonomous Simulation Testing

Several simulation testing were performed using QGroundControl (QGC) and Mission Planner to validate the autonomous navigational abilities of the UAV before integrated validation in real-world scenarios. Both QGC and Mission Planner are software systems that use MAVLink protocols to create a virtual setting for missions based on waypoints and provide configurations for those missions. The UAV's mission plan was uploaded and simulated within the software so the mission plan could be assessed in detail for path-finding accuracy, waypoint transitions, and command response. The tests involved varying mission-based scenarios, such as flight efficiency scenarios involving take-off, loitering, return-to-launch, and emergency failsafe scenarios, all under virtual wind and terrain profiles.

Telemetry logs monitored the UAV performance, and after conducting the missions using the UAV, key performance metrics like response latency, the reliability of mode switching, and CPU utilization of the onboard system were analyzed for robustness and stability of the systems before moving to live testing. These tests showed an accurate simulation result where the UAV was able to take off smoothly, follow the needed waypoint, and land safely, which confirms the perfect performance autonomous navigation node.

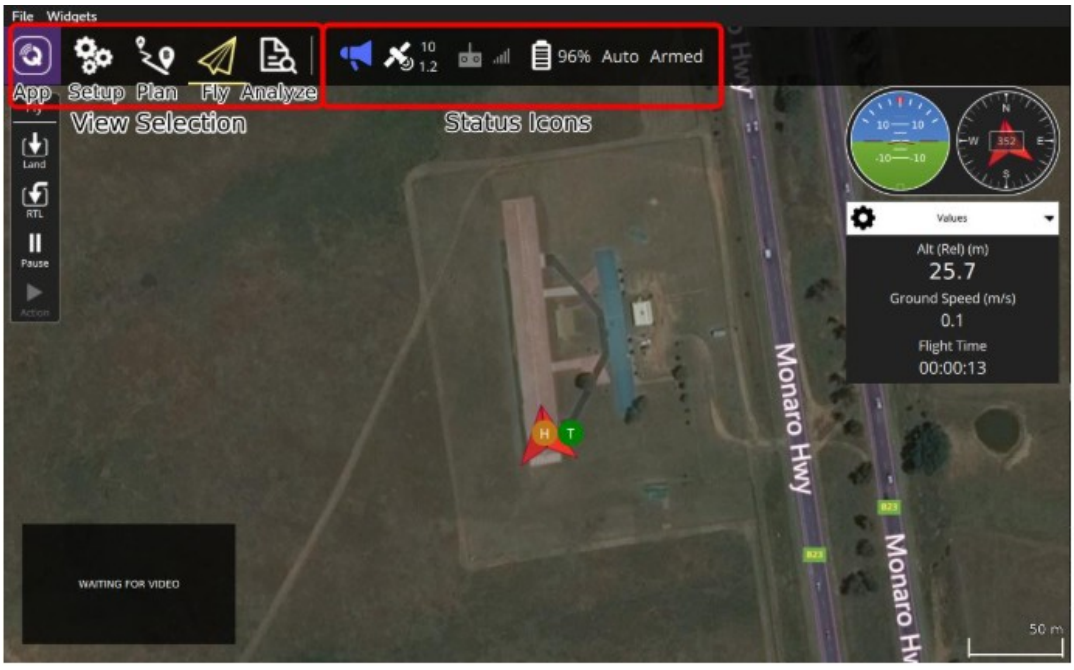

Figure 39 – Autonomous flight simulation

5.3.4 Real-world autonomous testing

After successful simulations, real-world flight tests were carried out to evaluate the UAV's autonomous navigation performance under actual conditions. As a safety precaution, initial tests were performed without propellers to prevent crashes, particularly in cases where the GPS accuracy was less than perfect or when signal noise and interference could affect navigation reliability. The tests begin by sending the mission from the ground station to take off 1 m and land, waiting to upload the mission to the UAV. Then, when armed, the UAV takeoff to the given altitude (1 m), observing the motors behavior if they are slowing down when landing. Next, after no props mission validation, the UAV was deployed in an open field and executed a planned mission, consisting of autonomous takeoff, multi-waypoint navigation, and return-to-launch using GNSS guidance and onboard sensors.

The UAV accomplished this with MAVROS nodes on the onboard Raspberry Pi communicating with the Pixhawk and translating ROS-based mission commands to the Pixhawk. The performance of the system was determined based on its accuracy in completing actions at the waypoints, steady flight path, and responses to obstacles while conducting automatic obstacle avoidance with the obstacle avoidance node. Records of telemetry, sensor data, and CPU load statistics were made during the entire mission, to be evaluated after flight. The test confirmed the system's viability to carry out advanced operations, such as fire detection and victim localization.

5.3.5 Obstacle avoidance

Testing obstacle avoidance using 360 LIDAR begins by checking the output data of the LIDAR through RVIZ simulation, which provides a visual data of the surrounding area, to ensure that the UAV frame or GPS stand does not appear in LIDAR detection and full 360° coverage with no gaps or noise artifacts. Confirm distance measurement accuracy by placing calibration targets such as flat walls 2 or 3 meters away and cross-checking RVIZ point-cloud ranges against ground-truth values ±2cm tolerance. Next, conduct incremental flight tests in a controlled static environment, command the UAV to autonomously approach obstacles while monitoring RVIZ's obstacle detection markers and the UAV's avoidance behavior, which will stop the UAV, move in a direction, then resume the autonomous flight.

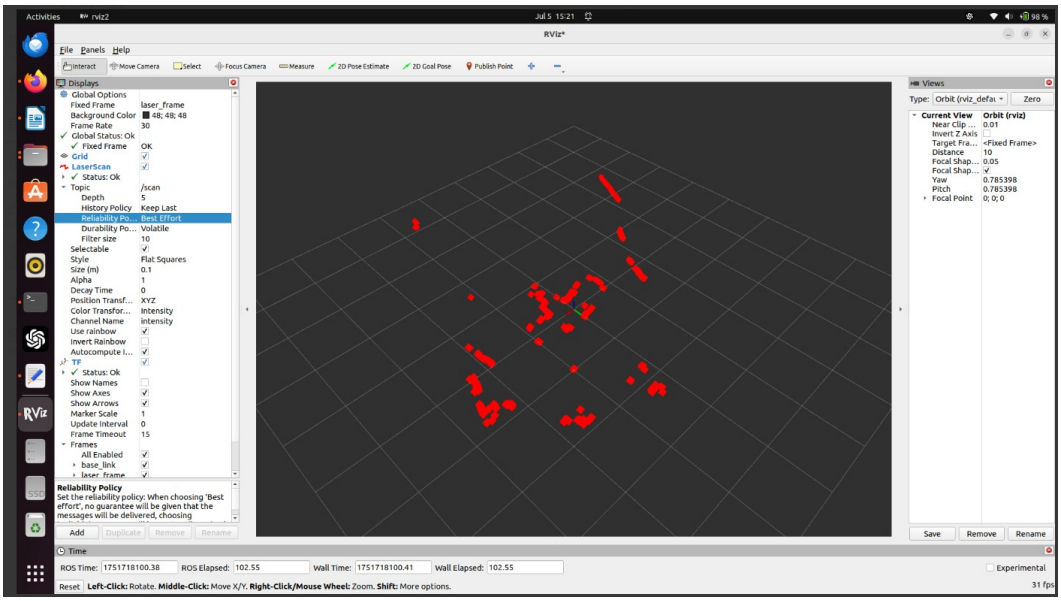

Figure 40 – RVIZ simulation

Conclusion

This chapter detailed the full mechanical and electrical assembly of the UAV, highlighting key design choices such as optimized weight distribution, vibration isolation, and passive thermal management to ensure stable in-flight performance. The integration of communication protocols, particularly MAVLink via MAVROS and Rosbridge enabled reliable real-time data exchange between the flight controller, companion computer, and ground station.

Multi-phase testing, including bench-level validation, sensor verification, simulated mission execution in QGroundControl, and real-world autonomous flights, confirmed the system's operational reliability. The UAV consistently achieved accurate waypoint navigation, real-time telemetry transmission, and effective obstacle avoidance, validating the robustness of the integrated ROS-based architecture and confirming the platform's readiness for deployment in field missions involving fire detection and victim localization.